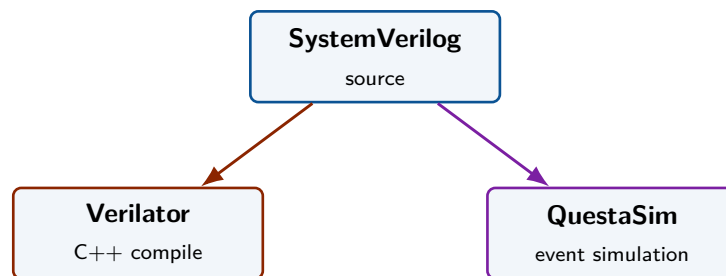


AN INTRODUCTION TO SYSTEMVERILOG

SystemVerilog for VHDL Designers

The Simulation Cookbook

Verilator vs. QuestaSim · Python and cocotb



Luigi Giuffrida, Alessandra Dolmeta

Department of Electronics and Telecommunications
Politecnico di Torino

Draft — September 17, 2026

This booklet is one of four companions: Design, Verification, Simulation (this one), and a complete worked Example. It covers Verilator compared with QuestaSim, and the four ways of connecting Python, including cocotb, to a testbench.

Preface

Sooner or later you will need to run a simulation without a QuestaSim licence in front of you, or with a golden model already written in Python that you would rather not reimplement in SystemVerilog. This booklet is about both situations.

This is the **Simulation Cookbook**, one of four companions covering SystemVerilog for VHDL Designers in full. It assumes the design and verification material of the *Design Cookbook* and the *Verification Cookbook*, and adds the tools that make a testbench runnable without a commercial licence, plus the four ways of driving one from Python.

What this booklet is

Chapter 1 is about Verilator, which is not an event simulator and should not be treated as one: it compiles a language subset to C++, it is two-state, and it has no route to UVM, but it is also the fastest thing you will run on a laptop and the best free linter for SystemVerilog by a wide margin. The chapter compares it directly with QuestaSim, feature by feature, so you know before you start which of the two a given testbench needs. Chapter 2 covers the four ways of connecting Python to a testbench, from a bare socket to cocotb, which lets the HDL testbench disappear entirely.

What this booklet is not

It is not a verification methodology text; the class-based testbench and UVM that these tools ultimately run are covered in the *Verification Cookbook*, and the design-side language they compile is covered in the *Design Cookbook*. Where this booklet needs a design to simulate, it uses the CORDIC rotator developed in full in the *Example Cookbook*.

The tools

Chapter B lists exactly what to install, on Linux, macOS or WSL2, and Chapter A gathers every command line used in this booklet onto one page. Nothing in Chapter 1 or Chapter 2 needs a licence.

How to Read This Booklet

Typographic conventions

- Monospaced text is code: an identifier, a keyword, a file name or a command.
- Coloured monospaced text marks a language keyword under discussion.
- Green monospaced text is a file in the accompanying source tree.
- Sans-serif names are tools: QuestaSim, Verilator, cocotb, GTKWave.

The three kinds of box

△ Trap: What a trap looks like

A trap marks a mistake you are statistically likely to make.

Recipe: What a recipe looks like

A recipe is a pattern to copy. When the text says “write it like this”, this is the box it is in.

What a note looks like

A note supplies background: where a construct came from, what the standard actually says, why a tool behaves the way it does. Skip these on a first reading and come back when something surprises you.

A rule of thumb is one line you should be able to recite from memory.

If your simulator is Verilator, or you have no licence at all

Read Chapter 1 from the start; Chapter B lists exactly what to install first. Everything in it works with free tools on your own machine.

If your golden model is already written in Python

Go straight to Chapter 2 and use cocotb. Every technique in the *Verification Cookbook*’s layered testbench works in any IEEE 1800 simulator, including Verilator; this booklet only adds Python as the language driving it.

A word on tools

This booklet assumes you have access to a commercial simulator, most likely QuestaSim, because that is what a university lab has and what an employer will hand you. It also assumes you would rather not depend on one. Chapter [B](#) lists what to install and how; Chapter [A](#) gathers every command line used in the booklet onto one page.

The companion booklets

This is the **Simulation Cookbook**. the *Design Cookbook* and the *Verification Cookbook* cover the language this booklet's testbenches are written in. the *Example Cookbook* runs every tool discussed here against one complete design.

Contents

Preface	v
How to Read This Booklet	vii
1 Verilator	1
1.1 What Verilator Is	1
1.2 Verilator and QuestaSim Are Different Machines	2
1.3 The Flow, Hands On	10
1.4 The Generated C++ API	12
1.5 Writing a C++ Testbench Properly	16
1.6 Running a SystemVerilog Testbench: <code>--binary</code> and <code>--timing</code>	18
1.7 A UVM-Shaped Verification Environment in C++	20
1.8 Coverage and Assertions	29
1.9 Linting as a First-Class Use	31
1.10 Where Verilator Does Not Reach	34
1.11 A Makefile for the Whole Chapter	34
1.12 Checklist	35
2 Python and the Testbench	37
2.1 Why Python belongs in a verification flow	37
2.2 Four ways to couple Python to a testbench	39
2.3 Offline vectors	40
2.4 Generated source	49
2.5 In-process, part 1: DPI-C	52
2.6 In-process, part 2: Verilator and <code>pybind11</code>	57
2.7 In-process, part 3: <code>cocotb</code>	59
2.8 Out-of-process co-simulation	66
2.9 Choosing	67
2.10 Putting it in CI	67
2.11 Checklist	70
A Command-Line Reference	73
A.1 Command lines	73

B Setting Up a Toolchain	75
B.1 The free toolchain	75
B.2 The commercial toolchain	76
B.3 A project layout that works	76
B.4 Continuous integration	77
B.5 Editor support	77
Further Reading	79
Index	81

Verilator

Verilator is not a simulator in the sense you learned with QuestaSim: it is a compiler that turns synthesisable SystemVerilog into C++, which you then build into your own program. This chapter explains what that changes, what it buys you and what it costs you; it takes you from a forty-line `main()` to a complete UVM-shaped verification environment written in C++, with coverage, assertions and lint; and it says honestly which parts of SystemVerilog the tool does not implement. Every command, every listing and every block of output in this chapter was executed on Verilator 5.020.

1.1 What Verilator Is

Every simulator you have used so far reads your VHDL, elaborates it into an internal data structure, and then runs an *event-driven kernel* over that structure: a queue of pending events, a notion of delta cycles, a resolution function on every signal. ModelSim and QuestaSim both work this way, whichever language you feed them. You interact with the kernel through a prompt (`vsim`), a waveform window and a Tcl interpreter.

Verilator does none of this. It reads synthesisable SystemVerilog and *writes C++ source code* that computes the same function. That C++ is then compiled by `g++` or `clang++` and linked into a program that you wrote. The design becomes a C++ class; its ports become public data members; advancing the simulation means calling a method.

The name is a contraction of *Verilog* and *translator*, which is exactly what the tool does. It was written by Wilson Snyder and has been developed in the open since 1994; it is dual licensed under the *LGPL version 3* and the *Artistic License 2.0*, so you may use it on proprietary designs and link the generated code into proprietary programs without any licence obligation on your RTL. There is no licence server, no seat count and no annual renewal. For a student that is not a small detail: it is the difference between verifying at home and verifying only in the laboratory.

“Verilator is a compiler” is the whole chapter

If you remember one sentence, remember this one. Almost every surprise in this chapter follows from it. There is no interactive prompt, because there is no kernel to talk to. There is no `$time` unless your program maintains one. There is no “restart” command, because restarting means running your executable again. And a syntax error is a *compile* error, reported before anything runs at all.

The flow has three stages, shown in Figure 1.1. First Verilator itself (`verilator`) reads the SystemVerilog and emits C++ into a directory called `obj_dir` by default. Then `make` compiles that C++ together with your own `main.cpp` and with the small Verilator runtime library into an ordinary executable. Finally you run the executable, which may write a waveform file in VCD or FST format.

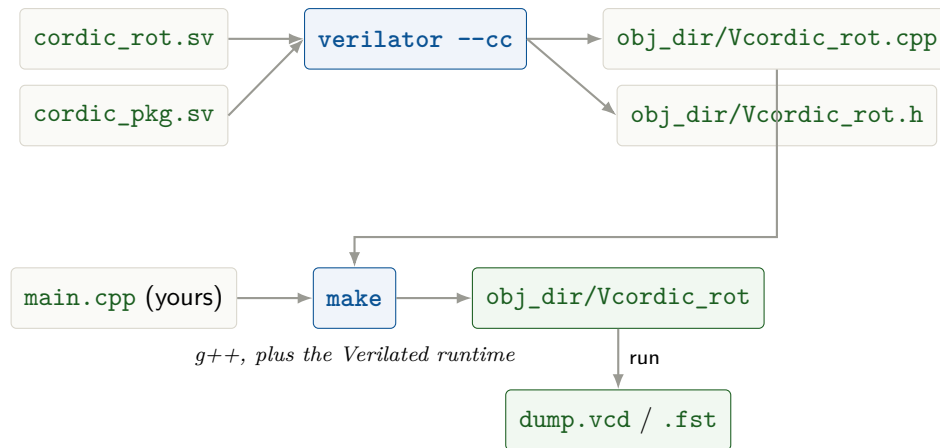


Figure 1.1. The Verilator flow. The middle stage is an ordinary C++ build; the last stage is an ordinary program.

1.2 Verilator and QuestaSim Are Different Machines

This section is the one to read slowly. Everything that goes wrong later goes wrong because a habit formed against an event-driven four-state simulator was carried unchanged into a compiled two-state one.

1.2.1 Event-driven versus statically scheduled

A VHDL simulator holds, at every point in time, a queue of pending signal updates and a set of processes waiting on events. When a signal changes, the kernel looks up its sensitivity list, wakes the processes that care, runs them, collects the new transactions, and repeats until no more delta cycles are pending. That machinery is what lets you write `wait for 3 ns`, model a transport delay, resolve two drivers on a bus, and stop the simulation at an arbitrary instant to inspect it. It also costs: every signal is an object, every update is an event, and every event goes through the queue.

Verilator throws the queue away. At compile time it analyses the whole design and produces a *static schedule*: a straight-line C++ function that evaluates every piece of combinational logic exactly once, in an order that respects the data dependencies, and then commits the sequential updates. The transformations that make this possible are the same ones a synthesis tool performs:

- **Hierarchy flattening.** Module instances are inlined. In the generated code the CORDIC's internal signals live in one flat structure named `Vcordic_rot__024root`; the module boundary has ceased to exist.
- **Constant propagation.** `ATAN_ROM` is computed once by Verilator, at elaboration, and appears in the C++ as literals.
- **Dead-logic elimination.** Anything that cannot affect an output or a traced signal is deleted.
- **Bit blasting and re-packing.** Narrow signals are packed into machine words; a 20-bit `idata_t` becomes a `uint32_t`.
- **Activity regions.** The generated `eval()` is split into blocks guarded by “did anything in this cone change?” flags, so a quiet part of the design costs a comparison and nothing more.

The consequence for you is simple and unforgiving: *the intermediate states that an event-driven simulator would show you may not exist at all*. If two `always_comb` blocks would, in Questa, briefly disagree during a delta cycle, Verilator has already reordered them so that the disagreement never occurs. That is usually what you want, because a synthesised netlist would not show the glitch either, but it means Verilator is a poor tool for studying race conditions.

△ Trap: Verilator will not reproduce a delta-cycle race

If your SystemVerilog has a genuine race – a testbench that drives a signal with a blocking assignment in the same time step in which the DUT samples it – Questa may show it and Verilator may not, or the other way round. Verilator detects some of these statically and reports UNOPTFLAT or MULTIDRIVEN; it cannot detect all of them. Race hunting belongs in an event-driven simulator. Use clocking blocks (the *Verification Cookbook*) so that the race cannot exist in the first place.

1.2.2 Two states, not four

This is the single most important warning in the chapter, so it gets a demonstration rather than an assertion.

VHDL's `std_logic` has nine values; SystemVerilog's `logic` has four: 0, 1, X and Z. Verilator's internal model has *two*. Every signal is a plain C integer; there is no room for an unknown. Consider a design with a classic reset bug – an accumulator that is used but never reset:

```

1 module resetbug (
2     input logic      clk_i,
3     input logic      rst_ni,
4     input logic [7:0] d_i,
5     output logic [7:0] sum_o
6 );
7     logic [7:0] acc;
8     logic      en;
9
10    always_ff @(posedge clk_i or negedge rst_ni) begin
11        if (!rst_ni) en <= 1'b0;    // en is reset ...
12        else      en <= 1'b1;
13    end
14
15    always_ff @(posedge clk_i) begin
16        if (en) acc <= acc + d_i;    // ... acc is not
17    end
18
19    assign sum_o = acc;
20 endmodule

```

Listing 1.1. A register that is used but never reset

A four-state simulator propagates the unknown initial value of `acc` through the adder and out of the module, and the testbench sees X. Here is the same testbench run under Icarus Verilog 12.0, which is event-driven and four-state, and then under Verilator 5.020:

```

$ iverilog -g2012 -s tb_resetbug -o rb.vvp resetbug.sv && vvp rb.vvp
sum_o after reset + 4 clocks = xxxxxxxx (x)
VERDICT: X detected -- the reset bug is visible

$ verilator --binary --timing --top-module tb_resetbug resetbug.sv -o rb
$ ./obj_dir/rb
sum_o after reset + 4 clocks = 00000011 (3)

```

```
VERDICT: sum_o is a clean number -- bug invisible
```

Verilator gives you a clean, plausible, entirely wrong number. Nothing in the run suggests that anything is missing. A scoreboard comparing against a golden model that also starts from zero would happily report *pass*.

△ Trap: A reset bug that Questa catches can hide in Verilator

Verilator is *X-optimistic*: an unknown becomes a definite value, usually zero, and the missing reset disappears. Four-state simulators are *X-pessimistic* in the opposite direction: an X on the select of a multiplexer poisons the output even when both inputs are equal, which produces false failures. Neither model is the silicon. The practical rule is: run reset and initialisation checks in a four-state simulator, and run bulk regression in Verilator.

Verilator offers three switches that soften the problem. They are worth knowing precisely, because their names are similar and their effects are not.

- `--x-assign 0|1|unique|fast` controls what an *explicit* 'x' in your source becomes. The default is `fast`, which lets the optimiser choose whatever is cheapest.
- `--x-initial 0|unique|fast` controls the initial value of a variable that has no reset. The default is `unique`.
- At *run* time, `+verilator+rand+reset+0|1|2` overrides the `unique` choice with all-zeros, all-ones or random, and `+verilator+seed+N` picks the random seed.

Measured on the counter of Listing 1.1:

```
$ ./obj_dir/rb                                     # default
sum_o after reset + 4 clocks = 00000011 (3)
$ ./obj_dir/rb +verilator+rand+reset+2 +verilator+seed+1
sum_o after reset + 4 clocks = 00000011 (3)
$ ./obj_dir/rb +verilator+rand+reset+2 +verilator+seed+2
sum_o after reset + 4 clocks = 00000010 (2)
$ ./obj_dir/rb +verilator+rand+reset+2 +verilator+seed+99
sum_o after reset + 4 clocks = 11111111 (255)
```

The result now depends on the seed, and *that* is the signal you can automate against.

Recipe: Catch missing resets without four-state simulation

Run every regression twice with `+verilator+rand+reset+2` and two different `+verilator+seed+` values, and require the two runs to produce byte-identical output. Any dependence on the seed is, by construction, a dependence on an uninitialised value. This costs one extra run and finds most of what X-propagation would have found.

The other missing state is Z. Verilator supports tri-state modelling only in limited forms and will usually report `UNSUPPORTED` or convert the net to a plain wire. Bidirectional pads and open-drain buses are not its territory.

1.2.3 No timing, unless you ask for it

In VHDL you write `wait for 5 ns`; in SystemVerilog you write `#5`. Verilator's default mode has no notion of time passing inside the model at all. Give it a delay without saying what to do about it and it refuses to proceed:

```
$ verilator --cc --main --exe --build --top-module delay delay.sv
%Error-NEEDTIMINGOPT: delay.sv:4:5: Use --timing or --no-timing to specify
                        how delays should be handled
    4 |      #10 a = 1'b1;
      |      ^
%Error: Exiting due to 1 error(s)
```

With `--no-timing` the delay is silently dropped, and Verilator tells you so:

```
%Warning-STMTDLY: delay.sv:4:5: Ignoring delay on this statement due to
                        --no-timing
```

With `--timing`, Verilator compiles delays, wait statements and event controls into C++20 coroutines and the program does the right thing:

```
$ verilator --binary --timing --top-module delay delay.sv -o dl
$ ./obj_dir/dl
a = 1 at 10
- delay.sv:6: Verilog $finish
```

`--binary` implies `--timing`

In 5.020, `--binary` is shorthand for `--main --exe --build --timing`: it generates a `main()` for you and produces an executable in one step. You can see the coroutine support in the compiler command line it generates, which carries `-fcoroutines` and links `verilated_timing.o`.

What this means for gate-level work is blunt: Verilator does not do it. A `specify` block is parsed and *silently ignored*; there is no SDF back-annotation, no pin-to-pin delay, no timing check, no `$setuphold`. Post-layout timing simulation stays in the commercial tool. Verilator's answer to gate-level verification is a different one: verify the RTL exhaustively and fast, and trust static timing analysis for the rest – which is, in practice, what industry does anyway.

1.2.4 How much of SystemVerilog does 5.020 actually implement?

Verilator started as a synthesis-subset tool and has been growing a verification subset for years. The claims below were each checked on this machine with a small test file; the error and warning text is quoted verbatim.

Classes, mailboxes, semaphores, fork/join: yes. The following file compiles and runs under `--binary --timing`:

```
1 module feat;
2   class packet;
3     rand bit [7:0] addr;
4     rand bit [7:0] data;
5     constraint c_addr { addr inside {[16:31]}; }
6     function string show();
7       return $sformatf("addr=%0d data=%0d", addr, data);
8     endfunction
9   endclass
10
11   mailbox #(packet) mbx = new(4);
12   semaphore          sem = new(1);
13
14   task automatic producer();
15     repeat (3) begin
```

```

16     packet p = new();
17     void'(p.randomize());
18     mbx.put(p);
19     #10;
20     end
21 endtask
22
23 task automatic consumer();
24     repeat (3) begin
25         packet p;
26         mbx.get(p);
27         sem.get(1);
28         $display("[%0t] consumed %s", $time, p.show());
29         sem.put(1);
30     end
31 endtask
32
33 initial begin
34     fork producer(); consumer(); join
35     $display("fork/join completed at %0t", $time);
36     $finish;
37 end
38 endmodule

```

Listing 1.2. Class, mailbox, semaphore and fork/join under Verilator 5.020

Constrained random: no. The first attempt to build Listing 1.2 fails, and the message is the important part:

```

%Warning-CONSTRAINTIGN: feat.sv:5:16: Constraint ignored (unsupported)
                        : ... note: In instance 'feat'
5 |     constraint c_addr { addr inside {[16:31]}; }
  |     ~~~~~
  |     ... For warning description see
  |     https://verilator.org/warn/CONSTRAINTIGN?v=5.020
%Error: Exiting due to 1 warning(s)

```

Add `-Wno-CONSTRAINTIGN` and it builds; `randomize()` then returns values that ignore the constraint completely:

```

$ ./obj_dir/feat
[0] consumed addr=196 data=156
[10] consumed addr=156 data=2
[20] consumed addr=2 data=228
fork/join completed at 30
- feat.sv:37: Verilog $finish

```

`addr` was constrained to `[16,31]` and came out as 196. There is no constraint solver in 5.020: `randomize()` is a fancy way of calling `$urandom` on every `rand` field.

△ Trap: `randomize()` succeeds and lies

It returns 1. It does not warn at run time. If your testbench relies on constraints for correctness – an address that must be aligned, an opcode that must be legal – Verilator will feed the DUT illegal stimulus and your scoreboard will report failures that are not design bugs. Guard the class with `'ifdef VERILATOR` and provide a hand-written randomiser, exactly as `tb_cordic_pkg.sv` does with its `randomize_fallback()`.

Covergroups: no. Not partially, not with warnings – the parser rejects them:

```
%Error-UNSUPPORTED: cov.sv:3:3: Unsupported: covergroup
    3 |   covergroup cg;
      |   ~~~~~
%Error-UNSUPPORTED: cov.sv:4:36: Unsupported: cover bin specification
%Error-UNSUPPORTED: cov.sv:4:12: Unsupported: cover point
```

Functional coverage in Verilator is done with `cover` property, which *is* supported (Section 1.8), or by counting in C++.

Virtual interfaces: yes, but not with clocking blocks. A plain virtual interface works:

```
1 interface simple_if(input logic clk);
2     logic [7:0] d;
3 endinterface
4
5 class drv;
6     virtual simple_if vif;
7     function new(virtual simple_if v); vif = v; endfunction
8     task automatic run();
9         for (int i = 1; i <= 3; i++) begin
10             @(posedge vif.clk);
11             vif.d <= 8'(i);
12         end
13     endtask
14 endclass
```

Listing 1.3. A virtual interface driven from a class

```
$ ./obj_dir/vif2
virtual interface OK: d=3 at 35
```

Add a clocking block and hand it out through a modport, as `cordic_if.sv` does, and 5.020 stops:

```
%Error-UNSUPPORTED: cordic_if.sv:70:16: Unsupported: Modport clocking
    70 |   modport drv (clocking drv_cb, output rst_n, input clk);
      |           ~~~~~
```

Remove the modport and wait directly on the clocking block through the virtual interface, and you hit an internal error rather than a clean message:

```
%Error: Internal Error: vif.sv:13:13: ../V3Width.cpp:2751:
      MemberSel of non-variable
```

△ Trap: Clocking blocks do not survive a virtual interface

The idiom `@(vif.drv_cb); vif.drv_cb.sig <= x;` is the backbone of a modern SystemVerilog testbench and of UVM drivers. In 5.020 it does not work through a `virtual` interface. You must either drive the raw signals with non-blocking assignments at `@(posedge vif.clk)`, or move the driver into C++ where the same discipline is enforced by the structure of the evaluation loop. Both routes are shown later in this chapter.

UVM: no. UVM needs the constraint solver, covergroups, `uvm_config_db` built on parameterised static class members, deep `$cast`, and a working `uvm_factory`. Several of those are absent. There is no supported way to run UVM on Verilator 5.020, and you should treat any claim otherwise with suspicion. Section 1.7 builds the missing structure in C++ instead.

Assertions: a useful subset. Immediate assertions work. Concurrent assertions with `|->`, `|=>`, `disable iff`, `$stable`, `$past`, `$rose` and `$fell` work. Sequence operators do not:

```
%Error-UNSUPPORTED: cordic_checker.sv:13:56: Unsupported:
    ## () cycle delay range expression
%Error-UNSUPPORTED: cordic_checker.sv:13:58: Unsupported:
    ## (in sequence expression)
```

A parsing trap worth knowing. Two constructs in the book’s own `tb_cordic_pkg.sv` stop the Verilator front end, and both are surprises:

```
%Error: tb_cordic_pkg.sv:81:5: Unknown verilator comment:
    '/*verilator does not implement the constraint solver, so the same
    class*/'
%Error: tb_cordic_pkg.sv:283:18: syntax error, unexpected STRING,
    expecting ')' or ','
```

The first is a *comment*. Any comment whose first word is `verilator` is treated as a metacomment, so a sentence that happens to begin “Verilator does not implement...” becomes a syntax error. The second is C-style adjacent string literal concatenation inside `$error(...)`. Here Verilator is right and the source was wrong: 1800-2017 has no rule that joins two adjacent string literals, and no conforming tool accepts it. The SystemVerilog spelling is the concatenation operator, `{"a", "b"}`, and that is what `tb_cordic_pkg.sv` now uses.

△ **Trap: Never begin a comment with the word “Verilator”**

```
// Verilator does not support X is a pragma, not a comment. Write // This tool does
not support X, or put the word anywhere but first.
```

1.2.5 The comparison, item by item

Table 1.1 summarises. Read the right-hand column as a statement about Verilator 5.020 specifically; the project moves quickly and several rows have changed in the last three releases.

1.2.6 Why it is fast, and where it is not

The speed comes from removing indirection, not from cleverness at run time:

1. There is no event queue, so a signal update is a store to a struct member, not an allocation and an insertion into a priority structure.
2. There is no per-signal sensitivity lookup, because the order of evaluation was decided at compile time.
3. The generated code is compiled by an optimising C++ compiler, which inlines, vectorises and keeps hot values in registers.
4. Activity flags mean that quiet regions cost one branch.
5. Two-state values pack into machine words, so a 20-bit datapath is one `uint32_t` operation rather than an array of four-state cells.

Numbers, measured for this chapter on a two-core Intel Xeon at 2.80 GHz with 7 GiB of RAM, Verilator 5.020, g++ 13.3, generated code compiled at `-O2`. Treat them as an order of magnitude, not as a benchmark: the CORDIC is a tiny design and your mileage will differ by a large factor in both directions.

Table 1.1. QuestaSim and Verilator compared. The Verilator column reflects version 5.020 as installed and tested for this chapter.

Aspect	QuestaSim	Verilator 5.020
Licence and cost	Commercial, node-locked or floating, per seat	LGPLv3 or Artistic 2.0, free, no server
Execution model	Event-driven kernel with delta cycles	Static schedule compiled to C++
VHDL support	Full IEEE 1076-2008	None; SystemVerilog and Verilog only
Mixed language	VHDL and SystemVerilog in one elaboration	Not possible
Value system	Four-state (and nine-state for <code>std_logic</code>)	Two-state; <code>X</code> becomes 0, 1 or random
Timing and delays	Full, including transport and inertial	Only with <code>--timing</code> ; ignored otherwise
Gate level and SDF	Full back-annotation and timing checks	Not supported; <code>specify</code> silently ignored
SVA	Complete, including sequences and local variables	Useful subset; no <code>##</code> sequence delay
Covergroups	Complete	Rejected as <code>UNSUPPORTED</code>
Constrained random	Full solver with <code>dist</code> , <code>solve...before</code>	No solver; <code>CONSTRAINTIGN</code> , values unconstrained
UVM	Supported and shipped	Not usable
Classes, mailboxes	Complete	Supported, including <code>fork/join*</code>
Virtual interfaces	Complete	Yes, but not with clocking blocks
DPI-C	Complete	Supported, import and export
VPI	Complete	Partial; enough for cocotb (Chapter 2)
Waveforms	WLF, VCD, FSDB	VCD and FST
Interactive debug	<code>vsim</code> prompt, breakpoints, force, GUI	None; use <code>gdb</code> on your own executable
Compile time	Seconds; incremental per file	Verilation fast, C++ build dominates
Run time	Baseline	Typically one to two orders of magnitude faster
Memory	Large per-signal structures	Compact; packed C integers
Multithreading	Limited	<code>--threads N</code> partitions the schedule
Coverage	Line, branch, expression, toggle, functional, GUI	Line, toggle, user; text and annotation only
Continuous integration	Needs a licence server in the runner	A container and a <code>make</code> target

```
verilation only (verilator --cc --exe --trace, 3 runs)    0.08 s each
C++ build of the model + testbench (make -j2, -O2)       5.50 s
C++ testbench, 200 000 transactions = 3 200 006 cycles   0.47 s
                                                         -> 6.7 Mcycle/s
SystemVerilog testbench, --binary --timing, same load    6.20 s
                                                         -> 0.54 Mcycle/s
```

Three lessons are hidden in those five lines.

First, *verilation is not the slow part*. Eighty milliseconds to read the CORDIC and write the C++; five and a half seconds for g++ to compile it. On a large design the ratio stays similar, which is why -j on the --build step matters and why people cache `obj_dir`.

Second, *compile time dominates for small work*. If your regression runs for two seconds, a five-second build makes Verilator slower end to end than starting vsim on an already-compiled library. Verilator wins when the simulation is long: a processor booting Linux, a DSP chain processing a million samples, a week-long random regression.

Third, *--timing costs about an order of magnitude*. The same DUT, the same number of cycles, driven from a SystemVerilog class-based testbench with coroutines rather than from C++, ran at 0.54 instead of 6.7 million cycles per second. Coroutine suspension and resumption is cheap compared with an event queue and expensive compared with a function call.

Waveform tracing has a cost of its own. Twenty thousand transactions, about 320 000 cycles, with every signal traced:

```
no tracing                0.17 s
VCD  (--trace)             0.23 s   106 MB
FST  (--trace-fst)         0.90 s    13 MB
```

Trace nothing by default. Add a command-line flag that switches tracing on, and use FST when the run is long enough that 100 MB of VCD is a problem.

1.3 The Flow, Hands On

1.3.1 The switches you will actually type

Three output modes matter.

- `--cc` emits C++ and nothing else. You supply `main()`. This is the mode for a C++ testbench and the one this chapter uses most.
- `--binary` emits C++, generates a `main()`, enables `--timing` and builds an executable. This is the mode for running a SystemVerilog testbench.
- `--sc` emits a SystemC module: the ports become `sc_in<>` and `sc_out<>` and the class derives from `sc_module`. Useful when the DUT must sit inside an existing SystemC virtual platform.

Around them:

- `--top-module <name>` names the top. Verilator can usually infer it, but says so noisily when there is more than one candidate; always pass it.
- `--exe file.cpp` says “this C++ file is part of the program”, and makes the generated makefile link an executable.

- `-Mdir <dir>` changes the output directory from `obj_dir`. Use it when you build several variants of the same design, as the chapter Makefile does.
- `--build` runs `make` for you; `-j N` parallelises it.
- `-CFLAGS "..."` and `-LDFLAGS "..."` are passed to the C++ compiler and linker.
- `-o <name>` names the executable.

1.3.2 Verilating the CORDIC

Assume the sources of the *Example Cookbook* have been copied into `rtl/`. The first command produces C++ only:

```
$ verilator --cc --top-module cordic_rot rtl/cordic_pkg.sv rtl/cordic_rot.sv
$ ls obj_dir/
Vcordic_rot.cpp      Vcordic_rot__Syms.cpp      Vcordic_rot__024root.h
Vcordic_rot.h        Vcordic_rot__Syms.h        Vcordic_rot__024root__Slow.cpp
Vcordic_rot.mk       Vcordic_rot_pch.h          Vcordic_rot_cordic_pkg.h
Vcordic_rot_classes.mk Vcordic_rot_cordic_pkg__Slow.cpp
```

The naming is regular and worth decoding once. `V<top>.h` and `V<top>.cpp` are the public wrapper: the class you instantiate. `V<top>__024root` is the flattened design; `__024` is the escaped form of `$`, because Verilator names the root scope `$root`. Files ending in `__Slow` contain code that runs once (construction, reset of internal state, coverage dump) and are compiled with `-Os` rather than `-O2`; `DepSet` in a file name marks a group of statements that share a dependency set. `V<top>.mk` is the makefile that `--build` invokes.

1.3.3 A forty-line testbench

Listing 1.4 is the smallest program that does something useful: it releases reset, presents one request to the CORDIC, waits for the response and prints it. Read `tick()`-free code once, so that the structure of the loop is visible before it is abstracted away.

```
1  #include "Vcordic_rot.h"
2  #include "verilated.h"
3  #include <cstdio>
4
5  int main(int argc, char** argv) {
6      VerilatedContext ctx;
7      ctx.commandArgs(argc, argv);
8      Vcordic_rot dut{&ctx};
9
10     dut.clk_i = 0;
11     dut.rst_ni = 0;
12     dut.req_valid_i = 0;
13     dut.rsp_ready_i = 1;
14     dut.req_x_i = 9949;      // XO_UNIT: seed for cos/sin
15     dut.req_y_i = 0;
16     dut.req_theta_i = 8579;  // ~pi/3 rad in Q2.13
17
18     for (int cyc = 0; cyc < 200; ++cyc) {
19         if (cyc == 4) dut.rst_ni = 1;      // release reset
20         if (cyc == 6) dut.req_valid_i = 1;  // present one request
21
22         dut.clk_i = 0;                      // low phase: settle the logic
23         dut.eval();
24         ctx.timeInc(5000);                  // 5 ns, in 1 ps precision units
25
26         bool req_hs = dut.req_valid_i && dut.req_ready_o;
```

```

27     bool rsp_hs = dut.rsp_valid_o && dut.rsp_ready_i;
28     int16_t x = (int16_t)dut.rsp_x_o, y = (int16_t)dut.rsp_y_o;
29
30     dut.clk_i = 1;                // rising edge
31     dut.eval();
32     ctx.timeInc(5000);
33
34     if (req_hs) dut.req_valid_i = 0;
35     if (rsp_hs) {
36         printf("t=%3lu cyc=%d  rsp_x=%6d  rsp_y=%6d\n",
37             (unsigned long)ctx.time(), cyc, x, y);
38         break;
39     }
40 }
41 dut.final();
42 return 0;
43 }

```

Listing 1.4. `main_first.cpp`: the smallest useful Verilator testbench

```

$ verilator --cc --exe --build -j 2 --top-module cordic_rot \
    rtl/cordic_pkg.sv rtl/cordic_rot.sv tb/main_first.cpp
$ ./obj_dir/Vcordic_rot
t=220000 cyc=21  rsp_x= 8190  rsp_y= 14190

```

The time stamp is in picoseconds, because `timeInc()` counts `timeprecision` units and `cordic_rot.sv` declares `'timescale 1ns/1ps`; 220000 ps is 220 ns, that is 22 half periods of 5 ns.

Seeded with $x_0 = X0_UNIT = 9949$ and $y_0 = 0$, the CORDIC returns $K \cdot 9949 \cdot (\cos \theta, \sin \theta)$, which is $(\cos \theta, \sin \theta)$ scaled by very nearly 2^{14} , because `X0_UNIT` was chosen as `round( $2^{14}/K$ )`. For $\theta = 8579/2^{13} = 1.0472$ rad the exact values are $K \cdot 9949 \cdot \cos \theta = 8191.2$ and $K \cdot 9949 \cdot \sin \theta = 14189.0$. The design is right to within two LSB, and it took twenty-one clock cycles: four of reset, two idle, one to accept, fourteen micro-rotations.

1.4 The Generated C++ API

1.4.1 Context, model, ports

`VerilatedContext` holds everything that a simulator kernel would normally own: the current time, the command-line arguments, the random seed, the coverage database, the “tracing was ever enabled” flag. You may create several, one per model, which is how you run two independent designs in one process.

```

1 VerilatedContext ctx;
2 ctx.commandArgs(argc, argv);    // parses +verilator+... plusargs
3 ctx.traceEverOn(true);         // must precede any trace() call
4 ctx.timeInc(5000);              // advance time by 5000 precision units
5 uint64_t now = ctx.time();
6 Vcordic_rot dut{&ctx};         // the model, bound to this context

```

If you omit the context, the model attaches to a global default one. That is fine for a single model and a nuisance for anything else; pass it explicitly.

Ports become public data members whose C type is chosen by width. The rule is fixed and worth memorising, because it is the first thing you get wrong:

A module with ports of every width generates exactly this:

```
VL_IN8(&a1,0,0);
```

Table 1.2. Verilator’s C types for packed signals.

Width	Verilator type	C type	Declared as
1 to 8	CData	uint8_t	VL_IN8
9 to 16	SData	uint16_t	VL_IN16
17 to 32	IData	uint32_t	VL_IN
33 to 64	QData	uint64_t	VL_IN64
65 and up	WData	array of uint32_t	VL_INW

```
VL_IN8(&a8,7,0);
VL_IN16(&a16,15,0);
VL_IN(&a32,31,0);
VL_INW(&a100,99,0,4);
VL_OUTW(&y128,127,0,4);
VL_IN64(&a64,63,0);
```

△ **Trap: Every port is unsigned, whatever the SystemVerilog said**

`rsp_x_o` is declared `data_t`, that is logic signed [15:0]. In C++ it is an `SData`, that is `uint16_t`. Reading it without a cast gives you the two’s complement bit pattern reinterpreted as a positive number:

```
rsp_y_o as SData : 51347
rsp_y_o as int16_t: -14189
```

Always cast a signed output to the matching signed C type, and always mask a signed input to its width when you write it. The SystemVerilog sign information does not cross the boundary.

For a width that is not a multiple of eight, the unused upper bits of the C type are zero on outputs, and Verilator requires them to be zero on inputs. A 20-bit input written as `0xFFFFFFFF` is undefined behaviour, not a truncation.

1.4.2 The evaluation model

This is where a VHDL intuition fails hardest, so it gets a numbered recipe.

`eval()` does one thing: it brings the model to a settled state, consistent with the current values of the input members. It is not “advance one cycle”. It is not “advance one delta”. There is no clock inside the model. *You* are the clock: you write `dut.clk_i = 1` and then call `eval()`, and the fact that `clk_i` went from 0 to 1 between two calls is what makes the `always_ff` blocks fire.

1. Write every input member that should change this cycle.
2. Call `eval()`. Combinational outputs are now valid for the *current* values of the state registers.
3. Read whatever you need to sample *before* the edge. This is the C++ equivalent of a clocking block’s default input `#1step`.
4. Set `clk_i = 1`.
5. Call `eval()` again. This is the rising edge: the `always_ff` blocks run and the state registers take their new values.
6. Advance the context clock with `timeInc()` and, if tracing, call `dump()` at each of the two time points.

Figure 1.2 draws one cycle of this.

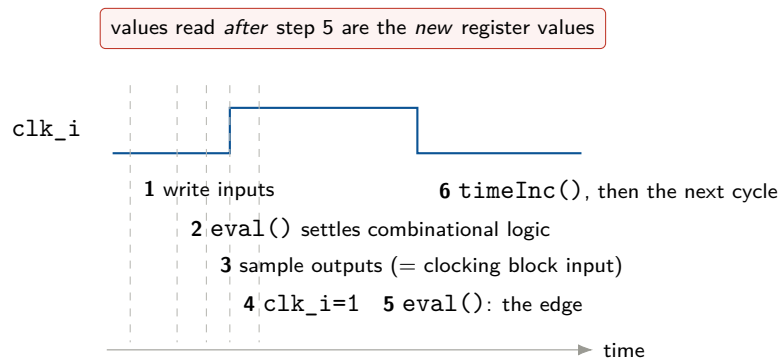


Figure 1.2. One clock cycle in a Verilator C++ testbench. There is no kernel: the six steps are six statements you write.

△ Trap: eval() is not “advance the clock”

The commonest beginner’s bug is a loop that toggles the clock but calls `eval()` only once per period, or that reads an output after the rising edge and treats it as the value that existed before it. In VHDL the distinction is made for you by `wait until rising_edge(clk)` and by the postponed update of signals. Here it is made by *where in your C++ you put the read*. Read handshake signals between steps 2 and 4; read registered data either there or after step 5, but be deliberate about which.

△ Trap: timeInc() counts in timeprecision units

`cordic_rot.sv` declares `'timescale 1ns/1ps`. A half period of 5ns is therefore `ctx.timeInc(5000)`, not `ctx.timeInc(5)`. Get it wrong and everything still runs, but every message the model prints is stamped with the wrong time. With `timeInc(5)` the DUT’s own assertion reports

```
[0] %Error: cordic_rot.sv:227: Assertion failed in TOP.cordic_rot:
    cordic_rot: |theta| > pi (theta = 30000)
```

and with `timeInc(5000)` the same failure reports [65000], which is 65 ns. The VCD timestamps are wrong in the same way.

Every C++ testbench in this chapter therefore advances time in picoseconds, `timeInc(5000)` per half period, and every simulated time printed here is in picoseconds. If you see `timeInc(5)` in a testbench that a colleague wrote, the design still simulates correctly; only the time stamps lie.

The other methods are quickly described. `eval_step()` and `eval_end_step()` split `eval()` in two, which matters only when several Verilated models are stepped together in one time slot: you call `eval_step()` on all of them, then `eval_end_step()` on all of them. `final()` runs the SystemVerilog `final` blocks and flushes internal state; call it once before the program exits, or coverage and `$display` output can be lost. `eventsPending()` and `nextTimeSlot()` exist only in `--timing` builds and let you write your own main loop for a design that has delays in it.

1.4.3 Tracing

Tracing must be enabled at verification time (`--trace` for VCD, `--trace-fst` for FST) *and* at run time. Forgetting the first gives a link error rather than a message about tracing, which is confusing the first time:

```
undefined reference to `VerilatedVcd::close()'
undefined reference to `VerilatedTrace<VerilatedVcd,
    VerilatedVcdBuffer>::dump(unsigned long)'
```

The run-time sequence is fixed: `traceEverOn(true)` before anything else, then construct the trace object, then `dut.trace(tfp, levels)`, then `tfp->open(name)`. After that, call `tfp->dump(time)` at every point you want a sample – typically twice per cycle, once for each clock phase – and `tfp->close()` at the end.

```
1 #if VM_TRACE_FST
2 # include "verilated_fst_c.h"
3 using TraceC = VerilatedFstC;
4 static const char* TRACE_FILE = "cordic.fst";
5 #else
6 # include "verilated_vcd_c.h"
7 using TraceC = VerilatedVcdC;
8 static const char* TRACE_FILE = "cordic.vcd";
9 #endif
```

`VM_TRACE`, `VM_TRACE_FST` and `VM_COVERAGE` are defined by the generated makefile, so one source file can serve both builds.

Recipe: Trace only what you need

`dut.trace(tfp, 99)` traces the whole hierarchy, including every `localparam` in `cordic_pkg.sv` – the VCD header for this tiny design already lists `ATAN_ROM[0..13]` and every format constant. Reduce the depth, or mark the interesting signals with `/* verilator public_flat */` and trace a shallow level.

1.4.4 The dump file is not the viewer

Nothing above draws a waveform. Verilator writes a *dump file*, and you open that file in a separate program. This is the one place where the free flow is visibly different from the commercial one: QuestaSim logs signals into its own `.wlf` database and shows them in its own wave window, so the simulator and the viewer are the same application. With Verilator they are not, and you choose both halves.

There are two dump formats.

VCD is the value-change dump defined by IEEE 1364. It is plain text, every tool on earth reads it, and it is enormous: a header listing every traced signal, then one line for every value change. A few thousand cycles of a small design is a few megabytes; a million cycles of a real one is a file you cannot open. Use it for short runs, for interoperability, and for anything you want to read with a script.

FST is the compressed binary format that came out of the GTKWave project. It is typically an order of magnitude smaller than the equivalent VCD and it opens in a fraction of the time, because a viewer can seek into it instead of parsing the whole thing. Use it for anything longer than a few thousand cycles. The measurements earlier in this chapter show the trade in the other direction: writing FST costs a little more simulation time than writing VCD, because the

compression happens as you go.

Both formats open in the same two viewers.

- GTKWave is the long-standing one. It reads VCD, FST and several other formats, it is in every distribution's package manager, and its interface is dated but complete: grouping, radix changes, saved signal lists in a `.gtkw` file, and a Tcl interface for scripting.
- Surfer is the newer one. It reads VCD and FST, the interface is modern, it starts quickly on large files, and there is a browser build as well as a native one, which is genuinely useful when you want to send somebody a waveform rather than a screenshot.

```
gtkwave cordic.fst          # or: gtwave cordic.vcd
surfer  cordic.fst

# QuestaSim, for comparison: the log and the viewer are the same tool
vsim -view vsim.wlf
```

Recipe: Save the signal list, not the screenshot

Both viewers can save the set of signals you have added, their order, their grouping and their radices, and reload it against a new dump. Do that once, and debugging the same block a week later starts from the picture you built rather than from an empty pane. In GTKWave it is **File>Write Save File**; the `.gtkw` belongs in the repository next to the testbench.

△ Trap: A gigabyte of waveform is not a debugging aid

`$dumpvars(0, top)` in SystemVerilog, or `dut.trace(tfp, 99)` in C++, dumps every signal in every instance below that scope, at every change. On the CORDIC it produces a file you can read; on anything with a memory in it, it produces tens of gigabytes and a viewer that will not open. Name the scope you care about — `$dumpvars(1, u_dut)` — or dump for a window of time by calling `$dumpoff` and `$dumpon` around the interesting cycles. The equivalent discipline in QuestaSim is to log a subtree rather than `log -r /*`.

1.5 Writing a C++ Testbench Properly

The forty-line version has all the ideas and none of the discipline. A usable testbench adds six things: a `tick()` helper so the clock protocol is written once, a reset sequence, a handshake driver that respects `ready`, a golden model, an error counter that becomes the process exit status, and tracing behind a command-line flag.

```
1 class CordicTb {
2 public:
3     VerilatedContext ctx;
4     Vcordic_rot*      dut;
5     TraceC*           tfp = nullptr;
6     uint64_t          cycles = 0;
7     unsigned          checked = 0, failed = 0;
8     double            worst = 0.0;
9
10    void openTrace(const char* name) {
11        ctx.traceEverOn(true);
12        tfp = new TraceC;
13        dut->trace(tfp, 99);
```

```

14     tfp->open(name);
15 }
16
17 // One clock period. Inputs written before the call are seen by the
18 // rising edge; outputs read after the call are the post-edge values.
19 void tick() {
20     dut->clk_i = 0; dut->eval();
21     if (tfp) tfp->dump(ctx.time());
22     ctx.timeInc(5000); // 5 ns at 1 ps precision
23
24     dut->clk_i = 1; dut->eval();
25     if (tfp) tfp->dump(ctx.time());
26     ctx.timeInc(5000);
27     ++cycles;
28 }
29 // The combinational outputs of the current cycle, before the edge.
30 void settle() { dut->clk_i = 0; dut->eval(); }
31
32 void reset(int n = 5) {
33     dut->rst_ni = 0;
34     for (int i = 0; i < n; ++i) tick();
35     dut->rst_ni = 1;
36     tick();
37 }
38
39 void check(const Txn& t) {
40     double th = double(t.theta) / (1 << QA);
41     double xr = double(t.x0) / (1 << QF);
42     double yr = double(t.y0) / (1 << QF);
43     double rx = K_GAIN * (xr * std::cos(th) - yr * std::sin(th));
44     double ry = K_GAIN * (xr * std::sin(th) + yr * std::cos(th));
45     double ex = std::fabs(double(t.x) - rx * (1 << QF));
46     double ey = std::fabs(double(t.y) - ry * (1 << QF));
47     if (ex > worst) worst = ex;
48     if (ey > worst) worst = ey;
49     ++checked;
50     if (ex > TOL_LSB || ey > TOL_LSB) {
51         ++failed;
52         printf("MISMATCH x0=%d y0=%d th=%d : got (%d,%d) "
53              "want (%.1f,%.1f) err %.2f/%.2f LSB\n",
54              t.x0, t.y0, t.theta, t.x, t.y,
55              rx * (1 << QF), ry * (1 << QF), ex, ey);
56     }
57 }
58 };

```

Listing 1.5. The core of `tb_cordic.cpp`

The golden model is deliberately written in real arithmetic, at the level of the specification, exactly as the SystemVerilog scoreboard of the *Verification Cookbook* is. A model that reimplements the micro-rotations would reproduce the design's bugs.

The handshake is the part that repays care. `req_ready_o` must be sampled *before* the edge that consumes the request, which is why `settle()` exists:

```

1  tb.dut->req_valid_i = 1;
2  tb.dut->req_x_i     = uint16_t(t.x0);
3  tb.dut->req_y_i     = uint16_t(t.y0);
4  tb.dut->req_theta_i = uint16_t(t.theta);
5  bool accepted;
6  do {
7      tb.settle(); // ready is now this cycle's value
8      accepted = tb.dut->req_ready_o;

```

```

9   tb.tick();                                // the edge that transfers it
10  } while (!accepted);
11  tb.dut->req_valid_i = 0;

```

△ Trap: Do not test ready after the edge

Writing `tick(); if (dut->req_ready_o) ...` tests the value that `ready` takes *after* the transfer, which for this DUT is 0 because it has moved to `ST_ROTATE`. The handshake then never appears to complete, `req_valid_i` stays high, and the DUT accepts the same request again the next time it is idle. The symptom is a testbench that hangs or that silently sends every transaction twice.

Running it:

```

$ verilator --cc --exe --build -j 2 --trace -CFLAGS -O2 \
  --top-module cordic_rot -Mdir obj_tb \
  rtl/cordic_pkg.sv rtl/cordic_rot.sv tb/tb_cordic.cpp -o tb_cordic
$ ./obj_tb/tb_cordic +n=200
-----
200 transactions, 0 failures, worst error 2.68 LSB
3206 clock cycles simulated
*** TEST PASSED ***
$ echo $?
0

```

The exit status is the interface to continuous integration. Return non-zero when anything failed, and never print “PASSED” from a run whose exit status is not zero.

1.6 Running a SystemVerilog Testbench: `--binary` and `--timing`

You do not have to abandon SystemVerilog for the testbench. With `--binary --timing`, Verilator compiles `initial` blocks, `#` delays, `fork/join*`, mailboxes and classes into coroutines and generates its own `main()`. The class-based testbench of the *Verification Cookbook* is close to running as it stands. Close, but not quite.

1.6.1 What has to change

The book’s `tb_cordic_pkg.sv` and `cordic_if.sv` need four edits before 5.020 accepts them, and each one teaches something:

1. **The modport that hands out a clocking block must go** (Unsupported: `Modport clocking`), and with it every `@(vif.drv_cb)` and `vif.drv_cb.sig <= x`. Replace the interface with one that carries plain signals, wait on `@(posedge vif.clk)` and drive with `<=`. You lose the race-freedom guarantee that the clocking block gave you, so the non-blocking assignment now matters: it is what keeps the driver’s writes in the NBA region, behind the DUT’s sampling.
2. **The comment beginning with the word “Verilator” must be reworded**, because it is parsed as a metacomment.
3. **Adjacent string literals in `$error` must be joined** into one string.
4. **`repeat (expr) @(...)` inside a `fork ...join_none` must become an explicit loop**, because of a lifetime restriction on the compiler-generated counter:

```
%Error-LIFETIME: tb_vlt_pkg.sv:62:13: Invalid reference:
Process might outlive variable `__Vrepeat1'.
```

The fix is three lines:

```
1 int unsigned stall = $urandom_range(3) + 1;
2 vif.rsp_ready <= 1'b0;
3 for (int unsigned k = 0; k < stall; k++) @(posedge vif.clk);
```

The parts that need no change at all are worth listing too, because they are the parts a VHDL designer would expect to be hardest: the classes, the inheritance, the mailbox #(cordic_txn) between generator and driver, the queue in the monitor, the fork ...join_none that starts four concurrent tasks, the real-valued golden model with \$cos and \$sin, and \$value\$plusargs. All of that runs.

1.6.2 Guarding with `ifdef VERILATOR

Verilator predefines the macro VERILATOR, so one source can serve both tools. The book's testbench already uses this for the two features that do not exist:

```
1 `ifndef VERILATOR
2   constraint c_theta { theta inside {[ -PI_EXT : PI_EXT]}; }
3 `endif
4
5 function void do_randomize();
6 `ifdef VERILATOR
7   randomize_fallback();
8 `else
9   if (!this.randomize()) $fatal(1, "randomization failed");
10 `endif
11 endfunction
```

The same pattern wraps the covergroup. Keep the guarded regions small and put the fallback next to the thing it replaces, so that the two cannot drift apart.

Recipe: Make the fallback the same distribution, not just legal

A hand-written fallback that picks θ uniformly in $[-\pi, \pi]$ is not the same stimulus as a dist that puts weight on the quadrant boundaries. If the constrained version is what you verify with, and the Verilator version is what you run nightly, write the corner cases into the fallback explicitly as directed transactions.

1.6.3 Running it

```
$ verilator --binary --timing -j 2 -CFLAGS -O2 --top-module tb_vlt_top \
  rtl/cordic_pkg.sv svtb/cordic_vif.sv rtl/cordic_rot.sv \
  svtb/tb_vlt_pkg.sv svtb/tb_vlt_top.sv -o tb_svtb
$ ./obj_svtb/tb_svtb +n=200
scoreboard: 200 checked, 0 failed
worst error: 2.82 LSB (tolerance 12 LSB)
*** TEST PASSED ***
simulated time: 33525000
- svtb/tb_vlt_top.sv:33: Verilog $finish
```

The whole layered testbench – generator, driver with random back-pressure, passive monitor, scoreboard with a trigonometric reference model – runs unmodified in structure on a free tool.

That is a genuinely useful result, and it is worth weighing against the two things you gave up: constrained random and functional coverage.

When to choose the SystemVerilog testbench over the C++ one

Choose SystemVerilog when the testbench must also run in QuestaSim (one source, two tools), when the protocol is complex enough that blocking tasks are much clearer than state machines, or when the people who will maintain it are RTL engineers. Choose C++ when the run is long and speed matters, when the golden model already exists in C or C++, or when you want to link the DUT into a larger software system.

1.7 A UVM-Shaped Verification Environment in C++

UVM is not available. That does not mean the *ideas* in UVM are unavailable: they are ordinary object-oriented design, and C++ expresses them with less ceremony than SystemVerilog does. This section builds a small, complete, header-only framework and runs it against the CORDIC. It is about 200 lines.

1.7.1 What UVM is actually for

Strip away the macros and the phasing and UVM does four things (the *Verification Cookbook*):

1. It gives components a **name and a place in a tree**, and calls `build_phase`, `connect_phase` and `run_phase` on every one of them in a defined order.
2. It provides **TLM ports** so a monitor can publish transactions without knowing who consumes them.
3. It provides a **factory**, so a test can substitute a different implementation of a component without editing the environment.
4. It provides a **configuration database**, so a test can change a parameter deep in the hierarchy without editing anything.

Points 3 and 4 are the reason UVM exists. Everything else is convenience. Figure 1.3 shows the hierarchy we will build, deliberately drawn the same way as the UVM hierarchy in the *Verification Cookbook*.

1.7.2 The framework

The whole framework is one header. The component base class carries a name, a parent, a list of children, and four virtual phase methods.

```

1 class Component {
2     public:
3         Component(std::string name, Component* parent)
4             : name_(std::move(name)), parent_(parent) {
5             if (parent_) parent_->children_.push_back(this);
6         }
7         virtual ~Component() = default;
8
9         std::string fullName() const {
10             return parent_ ? parent_->fullName() + "." + name_ : name_;
11         }
12     }

```

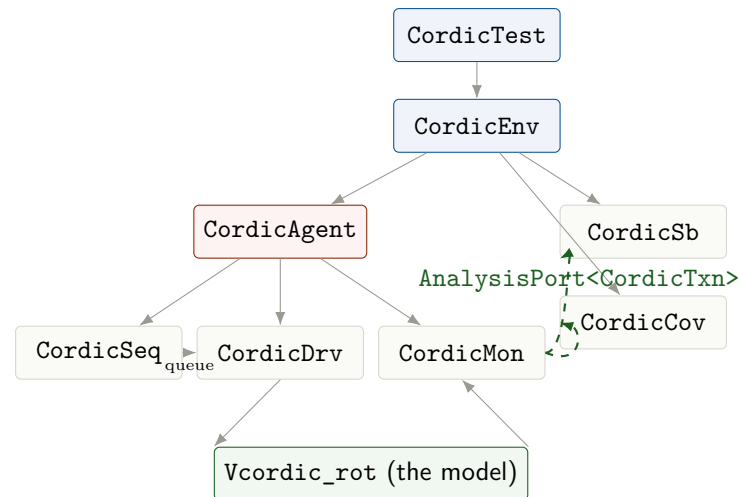


Figure 1.3. The C++ verification environment. Solid arrows are ownership, dashed green arrows are TLM connections. Compare with the UVM hierarchy in the *Verification Cookbook*: the boxes are the same.

```

13 // ---- the phases, overridden by the user -----
14 virtual void build() {}
15 virtual void connect() {}
16 virtual void tick() {}
17 virtual void report() {}
18
19 // ---- the phase walkers, called by the test -----
20 void buildAll() {
21     build();
22     // build() may have created children, so index by hand.
23     for (size_t i = 0; i < children_.size(); ++i) children_[i]->buildAll();
24 }
25 void connectAll() { // bottom-up, like UVM
26     for (auto* c : children_) c->connectAll();
27     connect();
28 }
29 void tickAll() {
30     tick();
31     for (auto* c : children_) c->tickAll();
32 }
33 void reportAll() {
34     for (auto* c : children_) c->reportAll();
35     report();
36 }
37 void info(const char* fmt, ...) const; // prints "[full.name] ..."
38 int cfgInt(const std::string& field, int dflt) const;
39
40 protected:
41     std::vector<Component*> children_;
42 private:
43     std::string name_;
44     Component* parent_;
45 };

```

Listing 1.6. `svt1m.h`: the component base class

The analysis port is nine lines, because C++ already has first-class callables. In UVM you would declare an `uvm_analysis_imp` with a macro so that several subscribers could each have a distinct write method; here a lambda does it.

```

1 template <typename T>
2 class AnalysisPort {
3 public:
4     using Sub = std::function<void(const T&);>;
5     void connect(Sub s) { subs_.push_back(std::move(s)); }
6     void write(const T& t) const { for (const auto& s : subs_) s(t); }
7 private:
8     std::vector<Sub> subs_;
9 };

```

Listing 1.7. The TLM analysis port

The factory maps a type *name* to a constructor, plus a table of overrides. Registration is one line in each component, hidden behind a macro that is the direct analogue of ‘uvm_component_utils.’

```

1 class Factory {
2 public:
3     using Ctor = std::function<Component*(const std::string&, Component*)>;
4     static Factory& get() { static Factory f; return f; }
5
6     void reg(const std::string& type, Ctor c) { ctors_[type] = std::move(c); }
7     void setTypeOverride(const std::string& orig, const std::string& repl) {
8         overrides_[orig] = repl;
9     }
10    Component* create(const std::string& type, const std::string& name,
11                     Component* parent) {
12        std::string t = type;
13        auto ov = overrides_.find(t);
14        if (ov != overrides_.end()) t = ov->second;
15        auto it = ctors_.find(t);
16        if (it == ctors_.end()) { printf("FATAL: no type '%s'\n", t.c_str());
17                                abort(); }
18        return it->second(name, parent);
19    }
20    template <typename T>
21    T* createAs(const std::string& type, const std::string& name,
22              Component* parent) {
23        return static_cast<T*>(create(type, name, parent));
24    }
25 private:
26     std::map<std::string, Ctor> ctors_;
27     std::map<std::string, std::string> overrides_;
28 };
29
30 template <typename T>
31 struct Registrar {
32     explicit Registrar(const std::string& type) {
33         Factory::get().reg(type, [](const std::string& n, Component* p) {
34             return static_cast<Component*>(new T(n, p));
35         });
36     }
37 };
38
39 #define SVTLM_REGISTER(TYPE) \
40     std::string typeName() const override { return #TYPE; } \
41     static inline svtlm::Registrar<TYPE> svtlm_reg_{#TYPE}

```

Listing 1.8. Factory and registration

The configuration store is a pair of `std::maps` keyed by “<component path>.<field>”, with “*” as a wildcard path. A component reads it during `build()` through `cfgInt()`, which is where UVM reads `uvm_config_db` too.

1.7.3 The hard part: blocking tasks versus callbacks

Here is the real difficulty, and it deserves to be stated plainly rather than designed around silently.

A SystemVerilog driver is a *blocking task*. It says “drive these pins, then wait until ready, then wait one more cycle, then drop valid”. The program counter of that task *is* the state of the protocol. Nothing else has to remember where the driver had got to.

A C++ method is not blocking. `tick()` is called, it returns, and every piece of state that must survive until the next call has to be stored in a member variable. There are two ways out.

(a) The tick model. Every component owns an explicit state machine and a `tick()` that advances it by one clock. This is portable to any C++ compiler, needs no library, and debugs well because the state is a named variable you can print. It costs you readability: the protocol is no longer a straight line of code.

```

1  class CordicDrv : public Component {
2      public:
3          SVTLM_REGISTER(CordicDrv);
4          CordicDrv(const std::string& n, Component* p) : Component(n, p) {}
5
6          CordicVif*      vif = nullptr;
7          std::deque<CordicTxn> q;
8          unsigned        stall_pct = 0, n_driven = 0;
9
10         void build() override {
11             stall_pct = cfgInt("stall_pct", 0);
12             rng_.seed(cfgInt("seed", 2));
13         }
14         void tick() override {
15             Vcortic_rot* d = vif->dut;
16
17             // --- request channel -----
18             if (st_ == IDLE) {
19                 if (!q.empty()) {
20                     const CordicTxn& t = q.front();
21                     d->req_valid_i = 1;
22                     d->req_x_i     = uint16_t(t.x0);
23                     d->req_y_i     = uint16_t(t.y0);
24                     d->req_theta_i = uint16_t(t.theta);
25                     st_ = DRIVING;
26                 } else {
27                     d->req_valid_i = 0;
28                 }
29             } else if (d->req_ready_o) { // handshake completes on this edge
30                 q.pop_front();
31                 ++n_driven;
32                 d->req_valid_i = 0;
33                 st_ = IDLE;
34             }
35
36             // --- response channel: random back-pressure -----
37             d->rsp_ready_i = (stall_pct == 0)
38                 || (int(rng_() % 100) >= int(stall_pct));
39         }
40         void report() override { info("drove %u transactions", n_driven); }
41
42     private:
43         enum State { IDLE, DRIVING } st_ = IDLE;
44         std::mt19937 rng_{2};

```

```
45 };
```

Listing 1.9. The driver as an explicit state machine

The monitor is written the same way and is where the analysis port lives:

```
1 void tick() override {
2     Vcortic_rot* d = vif->dut;
3     if (!d->rst_ni) { pending_.clear(); return; }
4
5     if (d->req_valid_i && d->req_ready_o) {
6         CORDIC_Txn t;
7         t.x0 = int16_t(d->req_x_i);
8         t.y0 = int16_t(d->req_y_i);
9         t.theta = int16_t(d->req_theta_i);
10        t.id = n_seen++;
11        pending_.push_back(t);
12    }
13    if (d->rsp_valid_o && d->rsp_ready_i) {
14        if (pending_.empty()) { info("ERROR response with no request"); return; }
15        CORDIC_Txn t = pending_.front();
16        pending_.pop_front();
17        t.x = int16_t(d->rsp_x_o);
18        t.y = int16_t(d->rsp_y_o);
19        ap.write(t); // TLM broadcast
20    }
21 }
```

Listing 1.10. The monitor publishes on the analysis port

The environment does the wiring, and this is where the TLM idea earns its keep: the monitor knows nothing about the scoreboard or the coverage collector.

```
1 void connect() override {
2     CORDIC_Sb* s = sb;
3     CORDIC_Cov* c = cov;
4     agt->mon->ap.connect([s](const CORDIC_Txn& t) { s->write(t); });
5     agt->mon->ap.connect([c](const CORDIC_Txn& t) { c->write(t); });
6 }
```

The clock loop is the only place that knows about both the model and the components. Note where `tickAll()` sits: after the settling `eval()` and before the rising edge, so every component sees exactly the pre-edge values, which is the semantics of a clocking block with default input `#1step`.

```
1 void run(VerilatedContext& ctx, unsigned max_cycles) {
2     Vcortic_rot* d = vif.dut;
3     d->clk_i = 0; d->rst_ni = 0;
4     d->req_valid_i = 0; d->rsp_ready_i = 0;
5     d->req_x_i = d->req_y_i = d->req_theta_i = 0;
6
7     for (unsigned c = 0; c < max_cycles; ++c) {
8         d->clk_i = 0;
9         d->eval(); // settle: pre-edge values are now valid
10        if (c == 5) d->rst_ni = 1;
11        if (c > 5) tickAll(); // every component sees this cycle
12        ctx.timeInc(5000); // 5 ns, in 1 ps precision units
13        d->clk_i = 1;
14        d->eval(); // the rising edge
15        ctx.timeInc(5000);
16        if (finished()) break;
17    }
```

```
18 }

```

Listing 1.11. The clock loop, in the test

△ Trap: A tick() that writes an input cannot then read its effect

Inside `tick()` you write `req_valid_i` and read `req_ready_o` in the same breath. The value you read was computed by the `eval()` that ran *before* `tickAll()`, so it does not include your write. For a well-formed valid/ready interface that is correct and desirable: it is exactly what a clocking block gives you. For a combinational handshake where `ready` depends on `valid`, it is wrong, and you must either call `eval()` again inside the tick or split the driver into two phases. Know which kind of interface you have.

(b) **C++20 coroutines.** The blocking style can be recovered. A coroutine suspends at `co_await` and resumes where it left off, which is precisely what a SystemVerilog task does at `@(posedge clk)`. Verilator itself uses this mechanism internally for `--timing`, and g++ 13 on this machine supports it with `-std=c++20 -fcoroutines`. A scheduler is three lines:

```
1 static std::vector<std::coroutine_handle<>> g_waiting;
2
3 struct AwaitClock {                      // == SystemVerilog's @(posedge clk)
4     bool await_ready() const noexcept { return false; }
5     void await_suspend(std::coroutine_handle<> h) const {
6         g_waiting.push_back(h);
7     }
8     void await_resume() const noexcept {}
9 };
10
11 struct Task {                             // == a SystemVerilog task
12     struct promise_type {
13         Task get_return_object() noexcept { return {}; }
14         std::suspend_never initial_suspend() noexcept { return {}; }
15         std::suspend_never final_suspend() noexcept { return {}; }
16         void return_void() noexcept {}
17         void unhandled_exception() { std::terminate(); }
18     };
19 };
20
21 static void resumeAll() {                  // one clock edge for every process
22     std::vector<std::coroutine_handle<>> ready;
23     ready.swap(g_waiting);
24     for (auto h : ready) h.resume();
25 }
```

Listing 1.12. A three-line cooperative scheduler

The driver then reads almost exactly like the SystemVerilog task it replaces:

```
1 Task driver(Vcortic_rot* d) {              // == task automatic run();
2     for (;;) {
3         while (g_queue.empty()) co_await AwaitClock{};
4         const Txn& t = g_queue.front();
5         d->req_valid_i = 1;
6         d->req_x_i      = uint16_t(t.x0);
7         d->req_y_i      = uint16_t(t.y0);
8         d->req_theta_i = uint16_t(t.theta);
9         // req_ready_o for THIS cycle is already visible: wait until it is
10        // high, then let one more edge pass -- that edge is the transfer.
11        while (!d->req_ready_o) co_await AwaitClock{};
```

```

12     co_await AwaitClock{};
13     d->req_valid_i = 0;
14     g_queue.pop_front();
15 }
16 }

```

Listing 1.13. The same driver, in blocking style

Starting three such processes is the C++ spelling of `fork ...join_none`:

```

1 driver(&d);
2 monitor(&d);
3 sequence(n);
4 for (; cyc < limit && g_checked < n; ++cyc) {
5     d.clk_i = 0; d.eval();
6     if (cyc == 5) d.rst_ni = 1;
7     if (cyc > 5) resumeAll();           // every process gets its clock edge
8     ctx.timeInc(5000);
9     d.clk_i = 1; d.eval();
10    ctx.timeInc(5000);
11 }

```

It compiles and it runs, and it produces the same numbers as the tick version:

```

$ verilator --cc --exe --build -j 2 -CFLAGS "-std=c++20 -fcoroutines -O2" \
  --top-module cordic_rot -Mdir obj_coro \
  rtl/cordic_pkg.sv rtl/cordic_rot.sv coro/main_coro.cpp -o tb_coro
$ ./obj_coro/tb_coro +n=200
coroutine TB: 200 checked, 0 failed, worst 2.68 LSB, 3206 cycles
*** TEST PASSED ***

```

Be honest about the cost. The forty lines above are the easy part; a real framework needs `join_any`, `join_none` with a handle you can kill, wait on an arbitrary condition, timeouts, and a way to propagate an exception out of a suspended coroutine. Lifetime is the sharp edge: a coroutine frame is heap-allocated and outlives the function that created it, so a reference captured before a `co_await` may dangle after it, and the compiler will not tell you. Measured on the same 200 000-transaction load, the coroutine version ran in 0.66 s against 0.47 s for the tick version – about 40% slower, still an order of magnitude faster than `--binary --timing`.

Write the tick model first. Reach for coroutines only when a protocol has so many states that the state machine has stopped being readable.

1.7.4 The factory and the config store in use

The test configures before it builds, exactly as a UVM test does in `new()` or in an early phase:

```

1 ConfigDb& cfg = ConfigDb::get();
2 cfg.setInt("n_txn", "n_txn", n);
3 cfg.setInt("stall_pct", "stall_pct", stall);
4 cfg.setInt("test.env.sb", "tol_lsb", 12);
5
6 if (strict) Factory::get().setTypeOverride("CordicSb", "CordicSbStrict");
7
8 Vcordic_rot dut{&ctx};
9 CordicTest test("test", nullptr);
10 test.vif.dut = &dut;
11 test.buildAll();
12 test.connectAll();
13 test.printTree();

```

Running with default settings:

```
$ ./obj_uvm/tb_uvm +n=200 +stall=25
[test.env.agt.seq      ] will generate 200 transactions
--- component tree ---
test (CordicTest)
  env (CordicEnv)
    agt (CordicAgent)
      seq (CordicSeq)
      drv (CordicDrv)
      mon (CordicMon)
      sb (CordicSb)
      cov (CordicCov)
--- run ---
--- report ---
[test.env.agt.drv      ] drove 200 transactions
[test.env.sb           ] 200 checked, 0 failed, worst 2.68 LSB (tol 12)
[test.env.cov          ] theta bins hit 4/5  [62 47 0 46 45]
*** TEST PASSED ***
```

Now the same binary with `+strict`, which registers a type override so that the factory builds a `CordicSbStrict` – a subclass with a tolerance of 2 LSB – wherever the environment asked for a `CordicSb`. Not one line of `CordicEnv` changes:

```
$ ./obj_uvm/tb_uvm +n=200 +strict
--- component tree ---
test (CordicTest)
  env (CordicEnv)
    agt (CordicAgent)
      seq (CordicSeq)
      drv (CordicDrv)
      mon (CordicMon)
      sb (CordicSbStrict)
      cov (CordicCov)
--- run ---
[test.env.sb ] MISMATCH id=33 theta=1276 got(-8109,-9602)
                                         want(-8107.0,-9602.8)
--- report ---
[test.env.agt.drv      ] drove 200 transactions
[test.env.sb           ] STRICT mode: 200 checked, 4 failed,
                        worst 2.68 LSB (tol 2)
*** TEST FAILED ***
$ echo $?
1
```

That is the whole point of a factory, demonstrated in eleven lines of framework code. The coverage collector reports `theta bins hit 4/5`: the bin for $\theta = 0$ exactly was never hit, which is the kind of thing functional coverage exists to tell you.

1.7.5 An honest comparison

When this is the right choice

Use a C++ environment when the reference model is already C or C++ (a DSP kernel, an instruction-set simulator, a codec), when the regression is long enough that speed dominates, when there is no licence, or when the design is small enough that UVM's overhead is larger than the design. Use UVM when the project is large, when several teams must reuse each other's agents, when a register model would otherwise be written

Table 1.3. The C++ framework of this chapter against UVM.

Feature	UVM 1.2 / IEEE 1800.2	This framework
Size to learn	About 400 classes and 200 macros	4 classes, 1 macro, 200 lines
Phasing	9 common phases plus 12 run-time sub-phases, with objections	4 phases, called explicitly by the test
Concurrency	Native blocking tasks	<code>tick()</code> state machines, or C++20 coroutines
TLM	Ports, exports, imps, FIFOs, analysis	<code>AnalysisPort<T></code> only
Factory	Type and instance override, by name and by type	Type override by name
Configuration	<code>uvm_config_db#(T)</code> , hierarchical wildcards	Two <code>std::maps</code> , one wildcard
Sequences	Sequence library, layering, arbitration, virtual sequences	A component that pushes into a queue
Constrained random Coverage	Full solver Covergroups, cross coverage, database merging	Whatever <code><random></code> gives you Counters and <code>cover property</code>
Register layer	<code>uvm_reg</code> , adapters, predictors, built-in tests	Nothing
Reporting	Severity, verbosity, ID filtering, catchers	<code>printf</code> with the component path
Reuse across projects	Real, and the industry expects it	Only what you carry yourself
Speed	Baseline	One to two orders of magnitude faster
Tool support	Every commercial simulator	Any C++17 compiler

by hand, or when the customer requires it. Do not use this framework as a substitute for learning UVM: learn UVM, then recognise that you can rebuild its useful 10% in an afternoon.

1.8 Coverage and Assertions

1.8.1 Collecting coverage

Three switches turn coverage on:

- `--coverage-line` counts how many times each basic block was entered.
- `--coverage-toggle` counts 0-to-1 and 1-to-0 transitions on every bit of every signal.
- `--coverage-user` counts cover property statements that you wrote.

`--coverage` is shorthand for all three. In a `--cc` build the database is not written automatically: your program must ask for it, which is one more consequence of “Verilator is a compiler”.

```
1 #if VM_COVERAGE
2     Verilated::mkdir("logs");
3     ctx.coveragep()->write("logs/coverage.dat");
4 #endif
```

The CORDIC has no cover points of its own, so we bind some onto it. A separate checker module keeps the RTL clean and is the same technique you would use in QuestaSim:

```
1 module cordic_checker (
2     input logic clk_i, rst_ni,
3     input logic req_valid_i, req_ready_o,
4     input logic rsp_valid_o, rsp_ready_i
5 );
6 // --- functional cover points (--coverage-user) -----
7 c_req_accept : cover property (@(posedge clk_i) disable iff (!rst_ni)
8                               req_valid_i && req_ready_o);
9 c_backpressure : cover property (@(posedge clk_i) disable iff (!rst_ni)
10                                rsp_valid_o && !rsp_ready_i);
11
12 // --- assertions (--assert) -----
13 a_reset_quiet : assert property (@(posedge clk_i) (!rst_ni) |-> !rsp_valid_o)
14     else $error("rsp_valid_o high during reset");
15 a_no_ready_drop : assert property (@(posedge clk_i) disable iff (!rst_ni)
16                                   (rsp_valid_o && !rsp_ready_i) |=> rsp_valid_o)
17     else $error("rsp_valid_o dropped before it was accepted");
18 endmodule
19
20 bind cordic_rot cordic_checker u_chk (.*);
```

Listing 1.14. Assertions and cover points bound onto the DUT

`bind` works, and `.*` works with it. Build with coverage on, run, and annotate:

```
$ verilator --cc --exe --build -j 2 --trace --assert \
    --coverage-line --coverage-toggle --coverage-user \
    --top-module cordic_rot -Mdir obj_cov \
    rtl/cordic_pkg.sv rtl/cordic_rot.sv cov/cordic_checker.sv \
    tb/tb_cordic.cpp -o tb_cov
$ ./obj_cov/tb_cov +n=300
$ verilator_coverage --annotate annotated logs/coverage.dat
```

```
Total coverage (72/118) 61.00%
See lines with '%00' in annotated
```

`verilator_coverage --annotate` writes a copy of every source file with a count in the left margin. A line prefixed `%000000` was never executed:

```
module cordic_checker (
%000001  input logic clk_i, rst_ni,
+009611  point: comment=clk_i
-000001  point: comment=rst_ni
 000600  input logic req_valid_i, req_ready_o,
+000600  point: comment=req_valid_i
+000601  point: comment=req_ready_o
);
 000300  c_req_accept : cover property (@(posedge clk_i) disable iff (!rst_ni)
+000300  point: comment=c_req_accept
%000000  c_backpressure : cover property (@(posedge clk_i) disable iff (!rst_ni)
-000000  point: comment=c_backpressure
```

Read that carefully, because it is coverage doing its job. `c_req_accept` fired 300 times, once per transaction. `c_backpressure` fired zero times: the testbench holds `rsp_ready_i` at 1 forever and has therefore never tested what happens when the consumer stalls. Adding four lines of random back-pressure to the testbench and re-running:

```
$ ./obj_cov/tb_cov +n=300 +bp
$ verilator_coverage --annotate annotated logs/coverage.dat
Total coverage (76/118) 64.00%
$ grep -A1 c_backpressure annotated/cordic_checker.sv
000120  c_backpressure : cover property (@(posedge clk_i) disable iff (!rst_ni)
000120                                     rsp_valid_o && !rsp_ready_i);
```

△ Trap: Toggle coverage on a reset signal is always 1

Look at the annotation above: `rst_ni` shows a count of 1, and it is marked - rather than +. Reset toggles once per simulation. That is not a coverage hole and no amount of stimulus will change it. Learn to recognise the points that are structurally unreachable before you start chasing a coverage percentage, and waive them in the `.vlt` file.

1.8.2 Assertions

`--assert` enables both the assertions you bound on and the ones the designer wrote inside the module. Without it they are compiled away entirely. `cordic_rot.sv` contains an immediate assertion on the input range; here is what happens when the testbench drives $\theta = 30000$, which is larger than $\pi \cdot 2^{13} = 25736$:

```
$ ./obj_as/tb_as # built with --assert
[65000] %Error: cordic_rot.sv:227: Assertion failed in TOP.cordic_rot:
        cordic_rot: |theta| > pi (theta = 30000)
%Error: rtl/cordic_rot.sv:227: Verilog $stop
Aborting...
$ echo $?
134

$ ./obj_na/tb_na # built without --assert
$ echo $?
0
```

Recipe: Always build the regression with --assert

The cost is small and the benefit is that every assertion the designers wrote becomes a test. Note also that a failing assertion calls `$stop`, which aborts the process; if you want the run to continue and count failures, use `$error` without an enclosing `assert`, or catch the abort in your build script.

The supported subset is the practically useful one: implication (`|->` and `!<=>`), `disable iff`, `$past`, `$stable`, `$rose`, `$fell`, `$onehot` and immediate assertions inside procedural code. The missing piece is sequences: `a ##1 b` and `a ##[1:3] b` are rejected as `UNSUPPORTED` in 5.020, which rules out most multi-cycle protocol properties. Where you need one, write it as a small checker RTL module with an explicit state machine and an immediate assertion; that also works in every other tool.

1.9 Linting as a First-Class Use

Many teams that never simulate with Verilator use it every day, because `verilator --lint-only -Wall` is the best free static checker for SystemVerilog and it takes under a second on a large design.

the *Design Cookbook* argued that VHDL's value comes largely from its type system: `std_logic_vector` does not silently become an `integer`, ranges are checked, and an unconstrained port is a compile error. SystemVerilog gives almost all of that up. Widths adapt silently, signedness is contagious in ways that surprise, an undeclared identifier becomes a one-bit wire, and an incomplete `case` produces a latch without comment. *Lint is the cultural replacement for the VHDL type checker.* A team that writes SystemVerilog without a lint gate in continuous integration has thrown away a safety net and put nothing in its place.

1.9.1 The warnings a VHDL designer meets first

The module in Listing 1.15 is a catalogue of first-week mistakes.

```

1  module sloppy (
2      input logic      clk,
3      input logic [7:0] a,
4      input logic [1:0] sel,
5      output logic [3:0] y,
6      output logic [7:0] q
7  );
8      logic [3:0] narrow;
9      logic [9:0] wide;
10
11     assign narrow = a;           // 8 bits into 4
12     assign wide   = a;           // 8 bits into 10
13
14     always_comb begin           // incomplete case
15         case (sel)
16             2'b00: y = 4'd1;
17             2'b01: y = 4'd2;
18             2'b10: y = 4'd3;
19         endcase
20     end
21
22     always_ff @(posedge clk) begin
23         q = a;                   // blocking in a clocked process

```

```

24  end
25
26  assign z = a[0];           // implicit net
27
28  logic [3:0] loop;
29  always_comb loop = {loop[2:0], narrow[3]};
30 endmodule

```

Listing 1.15. `sloppy.sv`: every mistake at once

```

$ verilator --lint-only -Wall --top-module sloppy sloppy.sv
%Warning-IMPLICIT: sloppy.sv:32:10: Signal definition not found,
                    creating implicitly: 'z'
%Warning-WIDTHTRUNC: sloppy.sv:14:17: Operator ASSIGNW expects 4 bits on
                    the Assign RHS, but Assign RHS's VARREF 'a'
                    generates 8 bits.
%Warning-WIDTHEXPAND: sloppy.sv:15:17: Operator ASSIGNW expects 10 bits on
                    the Assign RHS, but Assign RHS's VARREF 'a'
                    generates 8 bits.
%Warning-ALWCOMBORDER: sloppy.sv:36:15: Always_comb variable driven after
                    use: 'loop'
%Warning-UNUSED SIGNAL: sloppy.sv:10:15: Bits of signal are not used:
                    'narrow'[2:0]
%Warning-UNUSED SIGNAL: sloppy.sv:11:15: Signal is not used: 'wide'
%Warning-CASEINCOMPLETE: sloppy.sv:19:5: Case values incompletely covered
                    (example pattern 0x3)
%Warning-BLKSEQ: sloppy.sv:28:7: Blocking assignment '=' in sequential
                    logic process
                    : ... Suggest using delayed assignment '<='
%Error: Exiting due to 10 warning(s)

```

Two more appear only in slightly different code:

```

%Warning-LATCH: sloppy2.sv:4:3: Latch inferred for signal 'sloppy2.lat'
                    (not all control paths of combinational always assign
                    a value)
                    : ... Suggest use of always_latch for intentional latches
%Warning-UNOPTFLAT: sloppy2.sv:9:15: Signal unoptimizable: Circular
                    combinational logic: 'sloppy2.w'
                    Example path: sloppy2.w
                    Example path: ASSIGNW
                    Example path: sloppy2.w

```

What each one means to someone coming from VHDL:

WIDTHTRUNC / WIDTHEXPAND The assignment silently changed the width. In VHDL this is an error at analysis time. Treat both as errors and write explicit casts: `narrow = a[3:0];` or `wide = 10'(a);`.

IMPLICIT You used an identifier you never declared and Verilog made a one-bit wire out of it. VHDL has no such rule. Put `'default_nettype none` at the top of every file and this class of typo becomes an error.

CASEINCOMPLETE Not every value of the selector is covered. Combined with an `always_comb` that does not assign a default, this is a latch. In VHDL the others choice is compulsory in a case; SystemVerilog has no such requirement.

LATCH The inferred latch itself. `always_latch` declares that you meant it.

BLKSEQ A blocking = inside `always_ff`. This is the SystemVerilog form of the VHDL variable-versus-signal confusion, and it creates simulation-synthesis mismatches.

UNOPTFLAT Verilator found a combinational path from a vector back to itself that it cannot split

into independent bits. It is usually a real design smell and always a performance problem, because Verilator must then evaluate that region iteratively.

UNUSED SIGNAL Something is declared or driven and never read. Often harmless, often the fingerprint of a copy-paste error or a forgotten connection.

ALWCOMBORDER Inside one `always_comb` a variable is read before it is written. The block will still settle, but the intent is unclear.

1.9.2 Waiving warnings

Two mechanisms, and they have different politics. An inline pragma marks one place in one file:

```
1 // verilator lint_off WIDTHTRUNC
2 assign y = a;
3 // verilator lint_on WIDTHTRUNC
```

A configuration file, conventionally with the extension `.vlt`, keeps the waivers out of the RTL entirely and is passed on the command line like a source file:

```
// waivers.vlt -- project-wide lint waivers, kept out of the RTL.
`verilator_config
lint_off -rule UNUSED SIGNAL -file "sloppy2.sv" -match "'a'"
lint_off -rule UNOPTFLAT -file "sloppy2.sv" -match "'sloppy2.w'"
```

```
$ verilator --lint-only -Wall --top-module sloppy2 waivers.vlt sloppy2.sv
%Warning-LATCH: sloppy2.sv:4:3: Latch inferred for signal 'sloppy2.lat'
```

Both targeted warnings are gone and the latch remains, which is what you want.

Recipe: Waive in a file, not in the source

Inline `lint_off` is convenient and therefore dangerous: it spreads, it is invisible in review, and it is copied along with the code it was written for. Put waivers in a `.vlt` file with a comment saying why, so that the list of things your project has decided to ignore is one file that a reviewer can read.

The CORDIC of the *Example Cookbook* passes cleanly, which is the standard to hold yourself to:

```
$ verilator --lint-only -Wall --top-module cordic_rot \
    rtl/cordic_pkg.sv rtl/cordic_rot.sv
$ echo $?
0
```

Note that it does so partly by using two narrowly scoped pragmas in `cordic_pkg.sv` – `lint_off UNUSED PARAM` around constants that only testbenches read, and `lint_off UNUSED SIGNAL` around the deliberate truncation in `sat_narrow`. Both are documented in a comment on the line above. That is the right way to use an inline pragma: narrow, explained, and immediately turned back on.

Make `verilator --lint-only -Wall` a gate in continuous integration from the first commit. Retrofitting lint onto a 50 000-line design is a project; keeping it clean from the start is free.

1.10 Where Verilator Does Not Reach

Four things send you back to QuestaSim, and it is worth knowing them before you build a flow that assumes otherwise.

VHDL. Verilator does not read it. A mixed-language project — a VHDL block instantiated inside a SystemVerilog testbench, or the reverse — needs a simulator that reads both, and that means a commercial one. In QuestaSim you compile each language with its own compiler, `vcom -2008` and `vlog -sv`, into the same library and elaborate the two together; the port types that survive the boundary are the scalar and vector logic types, and anything cleverer needs a wrapper.

Four-state simulation. Verilator is two-state. A reset bug whose symptom is an X propagating through the design will not appear. Section 1.2.2 says what to do about it; the honest summary is that you run the reset tests somewhere else.

Covergroups and the constraint solver. Neither is implemented in 5.020. A testbench that depends on constrained random stimulus and functional coverage — which is to say, the one the *Verification Cookbook* teaches you to write — runs under Verilator with both features quietly absent.

UVM. It needs the class library, the DPI back end and language features Verilator does not have. There is no route.

Lint and regress with Verilator because it is fast and free; sign off with a four-state simulator because your silicon is not two-state.

1.11 A Makefile for the Whole Chapter

Everything in this chapter is driven from one file, and every target below was executed to produce the output printed above.

```
VERILATOR ?= verilator
JOBS      ?= $(shell nproc)

RTL       := rtl/cordic_pkg.sv rtl/cordic_rot.sv
CHK       := cov/cordic_checker.sv
COMMON    := --cc --exe --build -j $(JOBS) --top-module cordic_rot
CFLAGS_0 := -CFLAGS -O2

.PHONY: all first tb trace cov uvm coro svtb lint bench clean

all: first tb cov uvm coro svtb lint

# --- 1. the 40-line first example -----
first:
    $(VERILATOR) $(COMMON) -Mdir obj_first $(RTL) tb/main_first.cpp \
        -o tb_first
    ./obj_first/tb_first

# --- 2. the self-checking testbench -----
tb:
    $(VERILATOR) $(COMMON) --trace $(CFLAGS_0) -Mdir obj_tb \
        $(RTL) tb/tb_cordic.cpp -o tb_cordic
    ./obj_tb/tb_cordic +n=200

trace:
```

```

$(VERILATOR) $(COMMON) --trace-fst $(CFLAGS_0) -Mdir obj_fst \
    $(RTL) tb/tb_cordic.cpp -o tb_fst
./obj_fst/tb_fst +n=200 +trace

# --- 3. coverage and assertions -----
cov:
    $(VERILATOR) $(COMMON) --trace --assert \
        --coverage-line --coverage-toggle --coverage-user \
        -Mdir obj_cov $(RTL) $(CHK) tb/tb_cordic.cpp -o tb_cov
    ./obj_cov/tb_cov +n=300 +bp
    verilator_coverage --annotate annotated logs/coverage.dat

# --- 4. the UVM-shaped C++ environment -----
uvm:
    $(VERILATOR) $(COMMON) -CFLAGS "-O2 -I../uvmlike" -Mdir obj_uvm \
        $(RTL) uvmlike/main_uvmlike.cpp -o tb_uvm
    ./obj_uvm/tb_uvm +n=200 +stall=25

# --- 5. the coroutine variant -----
coro:
    $(VERILATOR) $(COMMON) -CFLAGS "-std=c++20 -fcoroutines -O2" \
        -Mdir obj_coro $(RTL) coro/main_coro.cpp -o tb_coro
    ./obj_coro/tb_coro +n=200

# --- 6. the SystemVerilog testbench under --binary --timing -----
svtb:
    $(VERILATOR) --binary --timing -j $(JOBS) $(CFLAGS_0) \
        --top-module tb_vlt_top -Mdir obj_svtb \
        rtl/cordic_pkg.sv svtb/cordic_vif.sv rtl/cordic_rot.sv \
        svtb/tb_vlt_pkg.sv svtb/tb_vlt_top.sv -o tb_svtb
    ./obj_svtb/tb_svtb +n=200

# --- 7. lint -----
lint:
    $(VERILATOR) --lint-only -Wall --top-module cordic_rot $(RTL)
    @echo "lint clean"

bench:
    ./obj_tb/tb_cordic +n=200000

clean:
    rm -rf obj_* annotated logs *.vcd *.fst

```

Recipe: Give every variant its own -Mdir

Coverage, tracing and `--timing` all change the generated code, so a single `obj_dir` would be rebuilt from scratch every time you switch targets. Separate directories cost disk and save minutes.

1.12 Checklist

- Verilator is a *compiler*. The DUT becomes a C++ class, the testbench is a `main()`, and there is no interactive prompt, no event queue for most signals, and no `$time` unless you keep one.
- **Never trust a Verilator run to catch a missing reset.** It is two-state and X-optimistic. Run reset checks in a four-state simulator (QuestaSim, Icarus), or run twice with `+verilator+rand+reset+2` and different `+verilator+seed+` values and require identical

output.

- `eval()` means “settle”, not “advance one cycle”. A clock edge is you writing `clk_i` and calling `eval()` again. Sample handshake signals *between* the settling `eval()` and the edge.
- `ctx.timeInc()` counts *timeprecision* units. Under ‘`timescale 1ns/1ps` a 5 ns half period is `timeInc(5000)`.
- Every port is unsigned in C++. Cast a signed output to `int16_t` or `int32_t` yourself; the SystemVerilog sign information does not cross the boundary. Widths map `CData/SData/IData/ QData/WData` at 8/16/32/64/65+ bits.
- `--trace` or `--trace-fst` must be given at verilator time as well as switched on at run time; otherwise you get a link error, not a message about tracing. `traceEverOn(true)` comes first.
- In a `--cc` build nothing writes the coverage database for you: call `ctx.coverage()->write("logs/coverage.cov")` before exit.
- In 5.020: classes, mailboxes, semaphores, `fork/join*`, virtual interfaces, `bind` and DPI-C all work. Covergroups, constraint solving, clocking blocks through a virtual interface, `##` sequence delays, SDF and UVM do not.
- `randomize()` returns 1 and ignores your constraints, after one `CONSTRAINTIGN` warning at build time. Guard the class with ‘`ifdef VERILATOR` and supply a hand-written randomiser.
- Never start a comment with the word “Verilator”: it is parsed as a metacomment and becomes a syntax error.
- When you need UVM’s structure and cannot have UVM, build the four things it is actually for: a named component tree with phases, an analysis port, a factory with type overrides, and a configuration store. Two hundred lines of C++ is enough.
- Blocking tasks do not exist in C++. Use `tick()` state machines first; use C++20 coroutines only when the state machine has become unreadable, and expect to pay about 40% in speed and much more in lifetime bugs.
- Run `verilator --lint-only -Wall` on every commit. It is the cultural replacement for the VHDL type checker (the *Design Cookbook*). Put ‘`default_nettype none` at the top of every file, treat `WIDTHTRUNC`, `WIDTHEXPAND`, `CASEINCOMPLETE`, `LATCH` and `BLKSEQ` as errors, and keep waivers in a `.vlt` file rather than inline.
- Verilator wins on long runs and loses on short ones: verilator of the CORDIC took 0.08s and the C++ build 5.5s, so a two-second regression is not where the tool pays for itself.
- **Four things send you back to QuestaSim:** VHDL and mixed-language elaboration, four-state semantics, covergroups and the constraint solver, and UVM. Section 1.10 says what to do about each.

Python and the Testbench

Your golden model is easier to write in Python than in SystemVerilog or VHDL, and you already know how to write it. This chapter answers the question that follows: how do you connect it to a simulation? It gives a taxonomy of four coupling styles – offline vectors, generated source, in-process calls, and out-of-process co-simulation – and works through each one with code that has actually been run against the CORDIC rotator. At the end you will be able to pick the right style for a homework exercise, a thesis accelerator or an industrial block, and to build the flow that goes with it.

2.1 Why Python belongs in a verification flow

A CORDIC reference model is three lines of `numpy`. A fixed point reference model of the same rotator, bit for bit, is about thirty lines of Python and about a hundred lines of SystemVerilog. That asymmetry is the whole argument, and it is worth being precise about where it comes from rather than repeating it as a slogan.

Python is good at five things a verification engineer needs:

- **Golden models.** Arbitrary-precision integers, real arithmetic, `numpy` arrays and `scipy` transforms are all one import away. Nothing in a Python model has a width, so the model never accidentally truncates. You add the truncation deliberately, which is exactly what a fixed point reference model is: a floating point algorithm plus a set of explicit rounding and saturation decisions.
- **Stimulus generation and corner-case enumeration.** Generating every angle that lands exactly on a quadrant boundary is a list comprehension. Enumerating the 2^{16} angles of a 16-bit input and filtering the interesting ones takes one line and a second of CPU time.
- **Post-processing.** A simulation produces numbers. Deciding whether those numbers are acceptable often means computing an error spectrum, a histogram or a plot – work that no HDL is designed for.
- **Code generation.** Register maps, ROM contents, decode tables, testbench skeletons, constant packages. Anything you would otherwise compute by hand and paste into a file.
- **Glue.** Makefiles, CI jobs, log parsing, result aggregation across a regression.

None of this is new. Verification teams have used Perl, Tcl and C for the same purposes for thirty years. What changed is that the numerical libraries moved to Python, and with them the people who write DSP and machine-learning hardware.

2.1.1 The honest counter-argument

The in-language model is sometimes the better choice, and a chapter that pretends otherwise would be selling something. SystemVerilog has `real` arithmetic, the `$atan/$sin/$exp` family

of real-valued system functions, and, since IEEE 1800-2005, `shortreal` as well. VHDL has `ieee.math_real`, which is older and arguably better specified. Both are perfectly capable of expressing a reference model.

Here is the arctangent table of the CORDIC computed at elaboration time in each language. The left pane is the real `cordic_pkg.vhd`; the right pane is the real `cordic_pkg.sv`.

VHDL

```
use ieee.math_real.all;

type atan_rom_t is
  array (0 to N-1) of iangle_t;

function build_atan_rom
  return atan_rom_t is
  variable r : atan_rom_t;
begin
  for i in 0 to N-1 loop
    r(i) := to_signed(integer(round(
      arctan(2.0 ** (-i))
      * 2.0 ** IQA)), IA);
  end loop;
  return r;
end function;

constant ATAN_ROM : atan_rom_t
:= build_atan_rom;
```

SystemVerilog

```
typedef iangle_t atan_rom_t [N];

function automatic atan_rom_t
  build_atan_rom();
  atan_rom_t rom;
  for (int i = 0; i < int'(N); i++)
    rom[i] = iangle_t'($rtoi(
      $atan(2.0 ** (-i))
      * (2.0 ** IQA) + 0.5));
  return rom;
endfunction

localparam atan_rom_t ATAN_ROM =
  build_atan_rom();
```

The two are the same idea: a pure function, called once by the tool, whose result becomes a constant. VHDL has `round`, so the intent is explicit; SystemVerilog has only `$rtoi`, which truncates towards zero, so the `+ 0.5` is doing the rounding by hand. That is the kind of detail that decides whether your model and your RTL agree in the last bit.

When the model is this small, keeping it in the RTL language is better: there is one source file, no build step, no generated artefact to go stale, and the constants are visible in the waveform viewer under their own names. Reach for Python when at least one of the following is true.

*Write the model in the RTL language if it fits on one screen and uses only arithmetic.
Write it in Python as soon as it needs arrays, a transform, a file, or a plot.*

There is a second, quieter reason to prefer Python once the algorithm is more than trivial: the model becomes the specification. `cordic_model.py` is thirty lines that anybody in the group can read, run and modify, including the person who has not yet learned SystemVerilog. When the architect changes the number of guard bits, the change happens in one file, the model is rerun in a second, the new error figure is printed, and only then does anybody touch RTL. A model buried inside a testbench package cannot play that role, because running it requires a simulator.

Recipe: Make the model runnable on its own

Give every Python model a `__main__` block that prints its parameters and its worst-case error. `cordic_model.py` does exactly that, and `python3 cordic_model.py` is the fastest sanity check in the whole flow – no simulator, no build, one second. When somebody reports that the hardware is wrong, run the model first.

Tool support for real-valued constant functions

Constant functions that call real-valued system functions are legal in IEEE 1800-2017 and IEEE 1076-2008, but support is uneven. Verilator 5.020 elaborates the `$atan` table above correctly. Icarus Verilog 12.0 does not: it cannot evaluate a constant function that returns an unpacked array, and it does not accept an unpacked array `localparam` at all. Synthesis tools vary more widely still; several accept `ieee.math_real` in a constant expression but reject the same expression inside a function used to initialise a ROM. This is the single strongest practical argument for generating such tables from Python: generated integer literals are accepted by everything.

2.2 Four ways to couple Python to a testbench

There are exactly four places where the two worlds can meet: in a file, at compile time, inside the simulator process, or across a process boundary. Figure 2.1 shows all four.

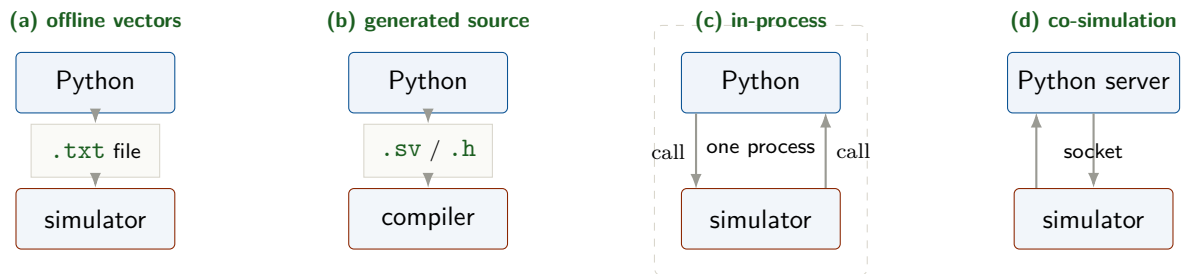


Figure 2.1. The four ways to couple a Python model to a simulation. They differ in *when* the coupling happens: (a) before the run, (b) before the build, (c) during the run in one process, (d) during the run across a process boundary.

Table 2.1 is the summary you should keep. The column that usually decides is the last one: can the Python side react to what the DUT does, cycle by cycle? Styles (a) and (b) cannot. That is not a defect; it is the reason they are cheap.

Table 2.1. Trade-offs of the four coupling styles.

	(a) offline	(b) generated	(c) in-process	(d) sockets
Run-time cost	none	none	μ s per call	μ s to ms per call
Debuggability	excellent: the file is the evidence	excellent: the output is source	mixed: two debuggers	poor
Reproducibility	excellent, if the file is archived	excellent	good, if versions are pinned	fragile
Setup complexity	lowest	low	high (DPI) or medium (cocotb)	high
Simulator portability	total	total	DPI: good; VPI: good; cocotb: good	total
Reacts cycle by cycle	no	no	yes	yes, slowly

2.3 Offline vectors

This is the workhorse. Master it first: it works with every simulator you will ever meet, it needs no libraries, it survives a change of tool vendor, and the intermediate file is a debugging artefact you can read with your eyes.

The flow has three steps. Python generates stimulus and expected responses into a file. The simulation reads the file, drives the DUT and writes what it observed into a second file. Python reads the second file and decides whether the design passed.

2.3.1 The generator

Listing 2.1 is the generator used throughout this chapter. It imports the golden model from `cordic_model.py` – the same module the cocotb testbench will import later, which is the point – and writes two files.

```

1 import argparse, math, random, subprocess, sys
2 from pathlib import Path
3 sys.path.insert(0, str(Path(__file__).resolve().parent))
4 import cordic_model as m
5
6 def hexw(value, width):
7     """Two's complement hex, `width` bits, no sign, no 0x prefix."""
8     return f"{value & ((1 << width) - 1):0{(width + 3) // 4}x}"
9
10 def corner_cases():
11     """Directed vectors: the angles a random generator will not hit."""
12     angs = [0.0, math.pi/2, -math.pi/2, math.pi, -math.pi,
13            math.pi/4, -math.pi/4, 1e-4, -1e-4]
14     out = [(m.XO_UNIT, 0, m.float_to_q(a, m.QA, m.AW)) for a in angs]
15     return out + [(32767, 0, 0), (-32768, 0, 0), (0, 0, 0)]
16
17 def random_cases(n, rng):
18     lim = m.float_to_q(math.pi, m.QA, m.AW)
19     return [(rng.randint(-6000, 6000), rng.randint(-6000, 6000),
20            rng.randint(-lim, lim)) for _ in range(n)]
21
22 def main():
23     args = parse_args() # --seed, --count, --outdir
24     rng = random.Random(args.seed)
25     cases = corner_cases() + random_cases(args.count, rng)
26
27     hdr = [f"seed={args.seed} count={len(cases)}",
28           f"DW={m.DW} QF={m.QF} AW={m.AW} QA={m.QA} N={m.N} ",
29           f"GL={m.GL} GM={m.GM}",
30           f"model=cordic_model.py rev={model_version()} ",
31           f"python={sys.version.split()[0]}",
32           "columns: x0 y0 theta xexp yexp (hex, 2's complement, 16 bit)"]
33
34     out = Path(args.outdir); out.mkdir(parents=True, exist_ok=True)
35     ftxt = open(out / "cordic_vec.txt", "w")
36     fhex = open(out / "cordic_vec.hex", "w")
37     for line in hdr:
38         ftxt.write("// " + line + "\n")
39         fhex.write("// " + line + "\n")
40     for (x0, y0, th) in cases:
41         xe, ye = m.cordic_rot(x0, y0, th)
42         cols = [hexw(v, 16) for v in (x0, y0, th, xe, ye)]
43         ftxt.write(" ".join(cols) + "\n")
44         fhex.write("".join(cols) + "\n") # one 80-bit word per line

```

```

45     ftxt.close(); fhex.close()
46     # $readmemh cannot report how many entries it filled: say so explicitly.
47     (out / "cordic_vec.n").write_text(f"{len(cases)}\n")

```

Listing 2.1. The vector generator (extract from `gen_vectors.py`).

Running it produces this:

```

$ python3 py/gen_vectors.py --seed 20250905 --count 200 --outdir vec

wrote 212 vectors to vec/cordic_vec.{txt,hex}

$ head -6 vec/cordic_vec.txt
// seed=20250905 count=212
// DW=16 QF=14 AW=16 QA=13 N=14 GL=2 GM=2
// model=cordic_model.py rev=no-git python=3.11.15
// columns: x0 y0 theta xexp yexp    (hex, two's complement, 16 bits)
26dd 0000 0000 3fff 0001
26dd 0000 3244 0001 3fff

```

2.3.2 Deciding the file format

Four decisions, and each of them will bite somebody if you get it wrong.

Hex or decimal? Hex. A hex field has a fixed width, which makes the file column-aligned and diffable, and it maps one to one onto the bit pattern on the wire. Decimal forces you to decide, in the file, whether the value is signed – and then to make the reader agree. Write hex, and treat the file as a picture of the pins.

One transaction per line. Never split a transaction across lines and never put two on one line. Line number equals transaction number is the most useful invariant a vector file can have: when the simulation reports a mismatch on transaction 137 you open the file at line 141 (137 plus the four-line header) and look at it.

A header. The first lines must record the seed, the parameter set and the model revision that produced the file. A vector file without a header is a liability: six months later nobody can tell whether it was generated for $N = 14$ or $N = 16$, and a testbench that reads it will happily report thousands of mismatches that mean nothing.

A comment convention. Use `//`. It is what `$readmemh` accepts, it is what a C++ `ifstream` loop can skip with two characters of test, and `std::textio` can skip it with one `read` of a character. A `#` convention would be more Pythonic and would break `$readmemh`.

2.3.3 Stimulus only, or stimulus and expected response?

The generator above writes both: the three inputs and the two expected outputs, on the same line. That is the right default, and it is worth being clear about why, because the alternative is common and defensible.

If the file contains only stimulus, the checking has to happen somewhere else – either in the HDL testbench, which then needs a model, or in Python afterwards, which then needs the results file and the stimulus file to be correlated by line number. If the file contains both, the HDL testbench is a pure comparator: it drives, it samples, it compares two integers. It needs no model, no real arithmetic and no knowledge of what a CORDIC is. That is a testbench a second-year student can maintain, and it is the same testbench whatever the design computes.

The cost is that the file is bigger and that it is now tied to one version of the model. Regenerate

it whenever the model changes – which is what the Make rule in Section 2.3 is for.

Two refinements are worth knowing. First, when the response is large (an image, a block of samples) put it in a separate file and keep the index in the stimulus file; a vector file you cannot open in an editor has lost most of its value. Second, when the transaction has optional fields, put a one-character command at the start of the line – W for write, R for read, # would clash with the comment convention – and dispatch on it in the reader. That turns the vector file into a small instruction stream, and it is how most directed-test frameworks start.

Recipe: The vector file header

Put the seed, every parameter the model depends on, the model file's revision and the Python version in the header, each on its own // line. Then make the testbench print the header it read at time zero. A regression log that contains the header of the vector file it consumed is self-documenting.

2.3.4 Reading it in SystemVerilog: \$readmemh

\$readmemh loads whitespace-separated hex tokens into successive entries of an array. It is the fastest thing to write and the easiest thing to get subtly wrong. The version below packs the five 16-bit fields of one transaction into a single 80-bit word, so that one array entry is one transaction.

```

1 module tb_vec_readmem;
2   import cordic_pkg::*;
3
4   localparam int MAXV = 4096;
5   localparam int VW    = 5*DW;          // 80 bits: x0 y0 theta xexp yexp
6
7   logic [VW-1:0] vec [0:MAXV-1];
8   int          nvec;
9
10  // How many entries did $readmemh actually fill? It will not tell us, so
11  // the generator writes the count into a companion file.
12  function automatic int read_count(input string name);
13    int fd, n;
14    fd = $fopen(name, "r");
15    if (fd == 0) $fatal(1, "cannot open %s", name);
16    if ($fscanf(fd, "%d", n) != 1) $fatal(1, "bad count file %s", name);
17    $fclose(fd);
18    return n;
19  endfunction
20
21  initial begin
22    if (!$value$plusargs("VEC=%s", vecfile)) vecfile = "vec/cordic_vec.hex";
23    nvec = read_count({vecfile.substr(0, vecfile.len()-5), ".n"});
24    $readmemh(vecfile, vec, 0, nvec-1);
25    $display("[tb] loaded %0d vectors from %s", nvec, vecfile);
26    ...
27    for (int i = 0; i < nvec; i++) begin
28      data_t x0, y0, xe, ye;
29      angle_t th;
30      {x0, y0, th, xe, ye} = vec[i];      // unpack one 80-bit word
31
32      @(negedge clk);
33      req_x = x0; req_y = y0; req_theta = th; req_valid = 1'b1;
34      do @(posedge clk); while (!req_ready);
35      @(negedge clk); req_valid = 1'b0;

```

```

36     do @(posedge clk); while (!rsp_valid);
37
38     if (rsp_x != xe || rsp_y != ye) errors++;
39   end
40   $display("[tb] %0d vectors, %0d mismatches", nvec, errors);
41   $display("[tb] %s", errors == 0 ? "PASS" : "FAIL");
42   $finish;
43 end
44 endmodule

```

Listing 2.2. Loading vectors with `$readmemh` (`tb_vec_readmem.sv`).

Built and run with Verilator:

```

$ verilator --binary --timing --top-module tb_vec_readmem \
    sv/cordic_pkg.sv sv/cordic_rot.sv sv/tb_vec_readmem.sv \
    -o Vtb_readmem --Mdir obj_readmem
$ ./obj_readmem/Vtb_readmem

```

```

[tb] loaded 212 vectors from vec/cordic_vec.hex
[tb] 212 vectors, 0 mismatches
[tb] PASS
- sv/tb_vec_readmem.sv:73: Verilog $finish

```

Note the unpacking line, `{x0, y0, th, xe, ye} = vec[i];`. A concatenation on the left-hand side is a SystemVerilog idiom with no VHDL equivalent: it splits a vector across several targets by width, left to right. It is the natural partner of a packed vector file.

△ Trap: `$readmemh` reads a short file in silence

The first version of this testbench filled the array with 'x' and then counted how many entries `$readmemh` had overwritten. Under Verilator that count came back as 4096 – the size of the array, not the size of the file – because Verilator is a two-state simulator and the 'x' initialisation had become zero. The run then drove 3884 all-zero transactions, which the DUT dutifully turned into all-zero responses, and reported:

```

[tb] loaded 4096 vectors from vec/cordic_vec.hex
[tb] 4096 vectors, 0 mismatches
[tb] PASS

```

A green result from a testbench that ran almost nothing. `$readmemh` does not report how many entries it filled, and if the file is shorter than the array the remaining entries simply keep their old value. In a four-state simulator they stay 'x' and you will usually notice; under Verilator they are zero and you will not. The fix is the one in Listing 2.2: have the generator write the count into a companion file and pass an explicit address range, `$readmemh(file, vec, 0, nvec-1)`.

Two more traps in the same system task. First, the address arguments are *addresses*, not counts, and they are inclusive: `$readmemh(f, m, 0, 9)` loads ten entries. Second, the file may contain `@` directives – a line `@1F` sets the load address to 'h1F' – and a stray `@` in a comment your generator wrote will silently relocate everything that follows. Generate the file, never hand-edit it, and never let a free-text field into it.

2.3.5 Reading it line by line

When the file has structure – several columns, optional fields, a per-line command – read it a line at a time. This is also the version to reach for when you want the file to stream: `$readmemh`

loads everything into memory before the simulation starts, which is fine for a thousand vectors and not fine for ten million.

```

1  int    fd_in, fd_out, code, nline = 0, nvec = 0, errors = 0;
2  string line, dir;
3
4  initial begin
5      if (!$value$plusargs("DIR=%s", dir)) dir = "vec";
6      fd_in  = $fopen({dir, "/cordic_vec.txt"}, "r");
7      if (fd_in == 0) $fatal(1, "cannot open the vector file");
8      fd_out = $fopen({dir, "/cordic_res.txt"}, "w");
9      ...
10     while (!$feof(fd_in)) begin
11         logic [15:0] x0, y0, th, xe, ye;
12         code = $fgets(line, fd_in);
13         nline++;
14         if (code <= 0) break;                // short read or EOF
15         if (line.len() < 2 || line.substr(0,1) == "//") continue;
16         code = $sscanf(line, "%h %h %h %h %h", x0, y0, th, xe, ye);
17         if (code != 5) begin
18             $display("[tb] line %0d: expected 5 fields, got %0d -- skipped",
19                     nline, code);
20             continue;
21         end
22         nvec++;
23         ... drive the DUT, wait for the response ...
24         $fdisplay(fd_out, "%04h %04h %04h %04h %04h",
25                     x0, y0, th, rsp_x, rsp_y);
26         if (rsp_x != xe || rsp_y != ye) errors++;
27     end
28     $fclose(fd_in); $fclose(fd_out);
29     $display("[tb] %0d lines, %0d vectors, %0d mismatches",
30             nline, nvec, errors);
31 end

```

Listing 2.3. Line-by-line reading with `$fgets/$sscanf` (`tb_vec_file.sv`).

```

$ ./obj_file/Vtb_file +DIR=vec
[tb] 217 lines, 212 vectors, 0 mismatches
[tb] PASS

```

`$sscanf` returns the number of fields it successfully converted, and that return value is the only line-format check you get. Test it. A file whose last line is truncated because the generator was interrupted will otherwise be read as a valid transaction with three of five fields left at their previous values.

△ Trap: `$feof` is not a look-ahead

`$feof` returns non-zero only *after* a read has hit the end of the file, exactly as in C. A `while (!$feof(fd))` loop therefore executes one extra iteration on an empty last read. That is why Listing 2.3 also checks the return value of `$fgets` and breaks. VHDL's `endfile` is a true look-ahead and does not have this problem, which is precisely why a VHDL designer forgets to add the check.

2.3.6 The same file in VHDL

For comparison, and because you will still be writing VHDL testbenches for some time, here is the same loop with `std.textio`. VHDL-2008's `hread` reads a hex value into a `std_logic_vector`

of matching width; before 2008 you needed `ieee.std_logic_textio` from a vendor library.

```

1  stim : process
2      file f      : text;
3      variable l   : line;
4      variable status : file_open_status;
5      variable c    : character;
6      variable ok   : boolean;
7      variable x0, y0, xe, ye : std_logic_vector(DW-1 downto 0);
8      variable th    : std_logic_vector(AW-1 downto 0);
9      variable nvec, errors : natural := 0;
10  begin
11      file_open(status, f, VEC, read_mode);
12      assert status = open_ok report "cannot open " & VEC severity failure;
13
14      wait for 33 ns;
15      rst_n <= '1';
16
17      while not endfile(f) loop
18          readline(f, l);
19          next when l'length = 0;
20          read(l, c, ok);                -- peek at the first character
21          next when not ok or c = '/';   -- skip '/' comments
22          l := new string(c & l.all);    -- put the character back
23
24          hread(l, x0, ok); next when not ok;
25          hread(l, y0);
26          hread(l, th);
27          hread(l, xe);
28          hread(l, ye);
29          nvec := nvec + 1;
30
31          drive_and_wait(x0, y0, th);    -- the usual handshake
32
33          if std_logic_vector(rsp_x) /= xe or std_logic_vector(rsp_y) /= ye then
34              errors := errors + 1;
35          end if;
36      end loop;
37      file_close(f);
38      report "vectors = " & integer'image(nvec) &
39            " mismatches = " & integer'image(errors);
40      assert errors = 0 report "FAIL" severity failure;
41      report "PASS";
42      wait;
43  end process;

```

Listing 2.4. The same vector file read with `std.textio` (`tb_vec_vhdl.vhd`).

```

$ vlib work
$ vcom -2008 vhdl/cordic_pkg.vhd vhdl/cordic_rot.vhd vhdl/tb_vec_vhdl.vhd
$ vsim -c -do "run -all; quit" tb_vec_vhdl

```

Everything the testbench prints comes from `report`, so the two lines that matter read the same whatever tool ran it, once the per-simulator prefix is stripped off:

```

vectors = 212 mismatches = 0
PASS

```

The interesting difference is not the syntax but the type discipline. `hread` knows how many hex digits to consume because the target has a width; `sscanf` does not, and will happily read six digits into a 16-bit variable and throw the top away. VHDL makes the format explicit and SystemVerilog makes it your problem – the same trade you meet everywhere else between the

two languages.

2.3.7 The same file in a Verilator C++ testbench

If your testbench is C++ (Chapter 1), the reader is an `ifstream` loop and the file format does not change at all. That is the whole point of the style: one file, four readers.

```

1 struct Vec { int16_t x0, y0, th, xe, ye; };
2
3 static int16_t hex16(const std::string& s) {
4     return static_cast<int16_t>(std::stoul(s, nullptr, 16));
5 }
6
7 static std::vector<Vec> load(const char* path) {
8     std::vector<Vec> v;
9     std::ifstream f(path);
10    if (!f) { std::fprintf(stderr, "cannot open %s\n", path); std::exit(2); }
11    std::string line;
12    while (std::getline(f, line)) {
13        if (line.size() >= 2 && line[0] == '/' && line[1] == '/') continue;
14        std::istringstream is(line);
15        std::string a, b, c, d, e;
16        if (!(is >> a >> b >> c >> d >> e)) continue;
17        v.push_back({hex16(a), hex16(b), hex16(c), hex16(d), hex16(e)});
18    }
19    return v;
20 }
21
22 // The driving loop is the same handshake as before: assert req_valid_i,
23 // tick until req_ready_o, tick once more, then tick until rsp_valid_o.
24 for (const Vec& v : vecs) {
25     dut->req_x_i = v.x0; dut->req_y_i = v.y0; dut->req_theta_i = v.th;
26     dut->req_valid_i = 1;
27     dut->eval();
28     while (!dut->req_ready_o) tick(); // stall until the DUT accepts
29     tick();                          // the accepting clock edge
30     dut->req_valid_i = 0;
31     dut->eval();
32     while (!dut->rsp_valid_o) tick();
33     if (static_cast<int16_t>(dut->rsp_x_o) != v.xe ||
34         static_cast<int16_t>(dut->rsp_y_o) != v.ye) ++errors;
35 }

```

Listing 2.5. The vector file in a Verilator C++ testbench.

```

$ ./obj_cpp/Vtb_cpp vec/cordic_vec.txt
[cpp] 212 vectors, 0 mismatches, 6788 half-cycles
[cpp] PASS

```

△ Trap: Signed values in a hex vector file

26dd 0000 cdbc 0001 c000 contains three negative numbers, and nothing in the file says so. Every reader must be told the width and must sign-extend itself. In C++ the cast is `static_cast<int16_t>(...)`; in Python it is an explicit `v - 65536 if v & 0x8000 else v`; in SystemVerilog it is `$signed(...)` or an assignment to a **logic signed** variable; in VHDL it is the `signed(...)` type conversion. Get it wrong on *one* side and the mismatch report will show you 52668 where you expected -12868, which at least is obvious. Get it wrong on *both* sides in the same way and the comparison still passes – and your tolerance

check on the floating point reference will be the only thing that catches it. Always print at least one negative expected value at time zero and check it by eye.

2.3.8 Checking the other direction

The simulation wrote `cordic_res.txt`. Python now reads it, compares against the *floating point* reference (not the bit-exact one – the bit-exact comparison already happened inside the simulation) and reports the error in LSBs.

```

1 import numpy as np
2 import matplotlib
3 matplotlib.use("Agg")          # no display in CI
4 import matplotlib.pyplot as plt
5 import cordic_model as m
6
7 def s16(v):
8     return v - 65536 if v & 0x8000 else v
9
10 def load(path):
11     rows = []
12     for line in Path(path).read_text().splitlines():
13         line = line.split("//")[0].strip()
14         if not line:
15             continue
16         rows.append([s16(int(t, 16)) for t in line.split()])
17     return np.array(rows, dtype=np.int64)
18
19 def main():
20     r = load(args.results)
21     x0, y0, th, xd, yd = (r[:, i] for i in range(5))
22
23     theta = th / float(1 << m.QA)
24     xf = m.K * (x0 * np.cos(theta) - y0 * np.sin(theta))
25     yf = m.K * (x0 * np.sin(theta) + y0 * np.cos(theta))
26
27     ex, ey = xd - xf, yd - yf          # error in LSB of the output format
28     err = np.maximum(np.abs(ex), np.abs(ey))
29
30     # The float reference does not saturate; the DUT does. Exclude the
31     # vectors that hit a rail, and say how many they were.
32     hi, lo = (1 << (m.DW - 1)) - 1, -(1 << (m.DW - 1))
33     sat = (xd >= hi) | (xd <= lo) | (yd >= hi) | (yd <= lo)
34     err = np.where(sat, 0.0, err)
35
36     print(f"vectors      : {len(r)}")
37     print(f"saturated    : {int(sat.sum())} (excluded)")
38     print(f"worst |error|    : {err.max():.3f} LSB")
39     print(f"tolerance       : {args.tol_lsb} LSB")
40     bad = np.flatnonzero(err > args.tol_lsb)
41     print(f"outside tol    : {len(bad)}")
42
43     if args.plot:
44         fig, ax = plt.subplots(figsize=(7, 3.2))
45         ax.plot(theta[~sat], ex[~sat], lw=0.8, label="x error")
46         ax.plot(theta[~sat], ey[~sat], lw=0.8, label="y error")
47         for s in (args.tol_lsb, -args.tol_lsb):
48             ax.axhline(s, ls="--", lw=0.7, color="0.4")
49         ax.set_xlabel("theta [rad]")
50         ax.set_ylabel("error [LSB of Q1.14]")

```

```

51     ax.legend(loc="upper right", fontsize=8); ax.grid(alpha=0.3)
52     fig.tight_layout(); fig.savefig(args.plot, dpi=150)
53
54     return 1 if len(bad) else 0

```

Listing 2.6. Post-processing and plotting (`check_results.py`).

Run over a 721-point sweep of the unit circle:

```

$ python3 py/check_results.py sweep/cordic_res.txt --plot sweep/err.png
vectors      : 721
saturated    : 0 (excluded)
worst |error| : 3.597 LSB
rms  error   : 1.115 LSB
tolerance    : 12.0 LSB
outside tol   : 0
plot         : sweep/err.png

```

[figure placeholder]

The error-versus-angle plot produced by `check_results.py`: two traces, x error and y error in LSBs of $Q1.14$, against θ from $-\pi$ to $+\pi$. Both stay inside a band of about ± 3.6 LSB, with the largest excursions near $\pm\pi/2$ and $\pm\pi$ where the coarse rotation stage runs and the residual angle is largest. The dashed lines at ± 12 LSB are the acceptance tolerance; the traces never approach them.

That number, 3.597 LSB, is the whole reason to keep the floating point reference around. The bit-exact model told you the RTL matches the model. The floating point reference tells you the *model* is a good enough approximation of a rotation – which is a different question, and the one your supervisor will actually ask.

△ Trap: A floating point golden model cannot be compared exactly

A bit-exact Python model and the RTL must agree exactly: any difference is a bug in one of them, and the check is `==`. A floating point reference and a fixed point DUT will *never* agree exactly, and comparing them needs a tolerance. Choosing that tolerance is a design decision, not a testbench detail:

- Derive it. For an N -step CORDIC the angle quantisation contributes about 2^{-N} radian and the datapath truncation about one LSB per stage in the worst case; for $N = 14$ with two guard bits the bound is a little under 5 LSB. The testbench in the *Verification Cookbook* uses 12 LSB, which leaves margin without hiding a real bug.
- Never tune it upward until the test passes. If you must, write down in the commit message why the bound moved.
- Exclude the cases the reference cannot model. The float reference does not saturate; the DUT does. Before that exclusion was added, two directed vectors with `x0 = 32767` produced a reported worst error of 21193 LSB and a red build. The vectors were fine; the comparison was wrong.

2.3.9 Reproducibility

A vector file is data, and data in a verification flow has to be reproducible. Four rules.

The seed is an input, not a constant. Pass it on the command line, default it to something fixed, print it, and put it in the file header. A regression that finds a bug with `-seed 4711` must be repeatable by typing `-seed 4711`.

Pin the versions. `random.Random(seed)` in CPython is stable across versions in practice, but numpy's legacy `np.random.seed` is explicitly not guaranteed to be, and `set` iteration order is not either. If a vector file must be byte-identical next year, use `random.Random`, sort anything that came out of a set or a dict, and record the Python version in the header.

Store the file, or store the generator – decide. Committing the vector file makes the repository self-contained and the history readable, at the cost of a binary-ish blob in every diff. Regenerating it keeps the repository small but means a broken generator breaks every historical commit. The usual compromise: commit a small directed set (a few hundred vectors, reviewable) and regenerate the large random set in CI.

Make the build depend on the generator. This is the one people forget.

```
# The vector file depends on the generator AND on the model: change either
# one and the vectors are rebuilt.
vec/cordic_vec.txt vec/cordic_vec.hex vec/cordic_vec.n: \
    py/gen_vectors.py py/cordic_model.py
    $(PYTHON) py/gen_vectors.py --seed $(SEED) --count $(COUNT) \
    --outdir vec
```

Every generated artefact must have a Make rule whose prerequisites include both the generator and everything the generator imports. Otherwise the day you fix the model is the day your regression starts lying to you.

2.4 Generated source

The second style moves the coupling one step earlier: Python writes source code, and the compiler consumes it. The advantage over offline vectors is that the result becomes a constant in the compiled design, so there is no run-time file access and no timing at all. The cost is a build step and a generated file that can go stale.

2.4.1 One generator, two outputs

Listing 2.7 emits the CORDIC constants both as a C++ header for the Verilator testbench and as a SystemVerilog package for the RTL. It uses `jinja2`, which is worth the dependency as soon as the template has more than one loop in it: the alternative is a function full of string concatenation that nobody can read.

```
1 from datetime import date
2 from jinja2 import Template
3 import cordic_model as m
4
5 BANNER = ("// GENERATED by gen_tables.py on {date} -- DO NOT EDIT.\n"
6          "// Source of truth: cordic_model.py (N={N}, IQA={IQA}).\n")
7
8 CPP = Template("""{{banner}}
9 #pragma once
```

```

10 #include <array>
11 #include <stdint>
12
13 namespace cordic_ref {
14
15 constexpr int    kN = {{N}}, kDW = {{DW}}, kQF = {{QF}};
16 constexpr int    kAW = {{AW}}, kQA = {{QA}}, kXOUnit = {{XO}};
17 constexpr double kGain = {{K}};
18
19 constexpr std::array<int32_t, kN> kAtan = {
20     {%- for v in atan %}
21         {{ v }}{% " ," if not loop.last %}
22     {%- endfor %}
23 };
24
25 } // namespace cordic_ref
26 """
27
28 SV = Template("""{{banner}}
29 package cordic_tbl_pkg;
30     localparam int unsigned N    = {{N}};
31     localparam int unsigned IQA = {{IQA}};
32     localparam int signed  XO_UNIT = {{XO}};
33
34     localparam int signed ATAN_ROM [{{N}}] = '{
35     {%- for v in atan %}
36         {{ v }}{% " ," if not loop.last %}
37     {%- endfor %}
38     }';
39 endpackage : cordic_tbl_pkg
40 """
41
42 def main():
43     out = Path(sys.argv[1] if len(sys.argv) > 1 else "gen")
44     out.mkdir(parents=True, exist_ok=True)
45     ctx = dict(banner=BANNER.format(date=date.today(), N=m.N, IQA=m.IQA),
46               N=m.N, DW=m.DW, QF=m.QF, AW=m.AW, QA=m.QA, IQA=m.IQA,
47               K=repr(m.K), XO=m.XO_UNIT, atan=m.ATAN)
48     (out / "cordic_tbl.h").write_text(CPP.render(**ctx))
49     (out / "cordic_tbl_pkg.sv").write_text(SV.render(**ctx))

```

Listing 2.7. Emitting a C++ header and a SystemVerilog package from one model (`gen_tables.py`).

The output is boring, which is the point:

```

$ python3 py/gen_tables.py gen && head -20 gen/cordic_tbl.h
// GENERATED by gen_tables.py on 2026-09-05 -- DO NOT EDIT.
// Source of truth: cordic_model.py (N=14, IQA=15).

#pragma once
#include <array>
#include <stdint>

namespace cordic_ref {

constexpr int    kN      = 14;
constexpr int    kDW     = 16, kQF = 14;
constexpr int    kAW     = 16, kQA = 13;
constexpr double kGain   = 1.6467602540312922;
constexpr int    kXOUnit = 9949;

constexpr std::array<int32_t, kN> kAtan = {
    25736,

```

```
15193,
8027,
...
```

2.4.2 A package, not an include

You can emit either a SystemVerilog package or a `.svh` file that gets ‘included into a module. Emit the package.

An ‘include is textual substitution done by the preprocessor. Include it in two modules and you get two copies of the constants, with no guarantee – and no check – that both compiled from the same generation. Include it twice into the same scope by accident and you get a duplicate declaration error whose message points at a line number in a file you did not write. A package is a named scope with a single elaboration: import it in ten places and there is still exactly one `ATAN_ROM`, and the tool will tell you if two versions of the package are in the compilation library. This is the same argument you already accept in VHDL, where nobody would dream of using textual inclusion instead of a package.

The one case for an ‘include is a fragment that is not a legal compilation unit on its own – a list of port connections, a block of declarations that must land inside a module. If you find yourself generating those, consider whether the thing you actually want is a parameterised module or an interface.

2.4.3 The arctangent ROM: generate or elaborate?

`cordic_pkg.sv` computes the arctangent table at elaboration with `$atan`, as shown at the start of this chapter. `gen_tables.py` computes the same table in Python and writes integer literals. Both are correct. Which one you should use depends entirely on your tools.

Verilator 5.020 handles the elaborated version:

```
$ ./obj_atan/Vtb_atan
ATAN_ROM = 25736 15193 8027 4075 2045 1024 512 256 128 64 32 16 8 4
X0_UNIT = 9949, HALF_PI = 51472
```

which is exactly the Python table:

```
$ python3 py/cordic_model.py
N = 14, K = 1.646760, 1/K = 0.607253, X0_UNIT = 9949
HALF_PI = 51472, PI = 102944
atan table: [25736, 15193, 8027, 4075, 2045, 1024, 512, 256, 128, 64, 32,
             16, 8, 4]
worst |error| over [-pi, pi] = 0.000233 (3.82 LSB)
```

Icarus Verilog 12.0 does not:

```
$ iverilog -g2012 -DSYNTHESIS -o /dev/null sv/cordic_pkg.sv sv/cordic_rot.sv
sv/cordic_pkg.sv:106: error: Array rom needs an array index here.
sv/cordic_pkg.sv:109: error: Unable to evaluate parameter ATAN_ROM value:
                      cordic_pkg.build_atan_rom()
2 error(s) during elaboration.
```

That is the deciding factor, and it is a mundane one. Keep the constant function if every tool in your flow – simulator, linter, synthesiser, formal tool – accepts it, because it removes a build step and keeps the design self-contained. Generate the table the moment one of them does not.

△ Trap: The generated file that nobody regenerated

The failure mode of style (b) is always the same. Somebody changes `N` in the model, runs the simulation, sees it pass, and ships. The generated package still holds the old table, because nothing forced it to be rebuilt. Two defences, and you want both:

- a Make rule with the generator and the model as prerequisites, as in Section 2.3;
- a CI job that regenerates into a scratch directory and diffs.

```
$ make gen-check
python3 py/gen_tables.py .gencheck
diff -r gen .gencheck || (echo "generated files are stale: run 'make gen'"; \
                           exit 1)
Files gen/cordic_tbl_pkg.sv and .gencheck/cordic_tbl_pkg.sv differ
generated files are stale: run 'make gen'
make: *** [Makefile:30: gen-check] Error 1
```

Also put `GENERATED - DO NOT EDIT` and the name of the generator in the first line of every emitted file. Somebody will edit it anyway, but at least the diff will show that they did.

2.4.4 In version control, or not?

Both answers are defensible and the argument is old. Keeping generated files out of the repository is cleaner in principle: one source of truth, no possibility of a stale artefact, no noisy diffs. Keeping them in makes the repository buildable by somebody who does not have your Python environment, makes code review of the generated RTL possible, and makes `git blame` on a constant meaningful.

For a thesis or a course project, commit them, and add the `gen-check` job above. Your reviewer will not have `jinja2`. For a large industrial flow with a curated build environment, generate them.

Register-map generation

Generating a constant table is the toy version of an idea the industry runs at scale. A register map – addresses, fields, access types, reset values – is written once in a machine-readable form and everything else is generated from it: the RTL register file, the C header for firmware, the UVM register model (the *Verification Cookbook*), the documentation, and the address decoder in the testbench. The formats you will meet are IP-XACT (IEEE 1685, XML, heavyweight, universally supported by commercial tools) and SystemRDL (Accellera, far more readable). On the open-source side, `peakrdl` reads SystemRDL and emits SystemVerilog, VHDL, C headers, HTML and a UVM model; `corsair` does something similar from JSON or YAML. If your design has more than about ten registers, learn one of these instead of maintaining four copies of the same table by hand.

2.5 In-process, part 1: DPI-C

The Direct Programming Interface is the standard way for SystemVerilog to call C and for C to call SystemVerilog. It is defined in IEEE 1800-2017 clauses 35 and H, it is supported by every commercial simulator and by `Verilator`, and it is the mechanism underneath most of the Python integrations you will meet. VHDL has no equivalent in the standard: VHPI (IEEE 1076-2008 clause 22) is a procedural interface, not a foreign-function interface, and the practical

route in VHDL is the vendor-specific `foreign` attribute, whose spelling, calling convention and capabilities differ from one tool to the next.

2.5.1 Declaring an import

```

1 // Pure: same inputs give the same result, no side effects, no access to
2 // the design. The tool may cache the result or elide the call.
3 import "DPI-C" pure function real c_cordic_k();
4
5 // Context: the function needs to know which SystemVerilog scope called
6 // it, because it will call back in (or use svGetScope).
7 import "DPI-C" context function void c_cordic_ref(
8     input int x0, input int y0, input int theta,
9     output real xr, output real yr);
10
11 // Neither: the default. The function may have side effects but may not
12 // call back into SystemVerilog.
13 import "DPI-C" function void c_show_types(
14     input logic [31:0] l, input bit [31:0] b);
15
16 // Export: make a SystemVerilog function callable from C.
17 export "DPI-C" function sv_note;
18 function void sv_note(input string msg);
19     $display("[SV] note from C: %s", msg);
20 endfunction

```

Listing 2.8. DPI import and export declarations.

`pure` and `context` are the two qualifiers that matter. A `pure` function must depend only on its arguments, must not touch the design, must not do I/O, and must not call back into SystemVerilog. In exchange, the simulator may call it once and reuse the answer, or skip the call entirely if the result is unused. A `context` function is given the calling scope, which it needs if it wants to call an exported task or read a signal through `svGetScope`/`svSetScope`. Marking something `pure` that is not is a genuinely nasty bug, because the symptom is a stale value rather than a crash.

Two parts of the interface deserve a sentence each because they are where the examples in vendor documentation stop. `svGetScope()` returns an opaque handle to the SystemVerilog scope a `context` function was called from, and `svSetScope(h)` makes that scope current before you call an exported task. You need the pair when the C side must call back into a *specific* instance – a driver in one agent rather than another – and the usual pattern is to have each instance register its scope in an `initial` block, store the handle in a C table, and set it before each callback. Open arrays are the other gap: an argument declared `input int arr []` on the SystemVerilog side arrives as an `svOpenArrayHandle`, and you must call `svSize`, `svLow`, `svHigh` and `svGetArrElemPtr1` to get at the elements rather than indexing a C array. Sized arrays (`input int arr [8]`) arrive as an ordinary pointer and are far simpler; prefer them unless the size genuinely varies.

2.5.2 A working example

The C side computes the CORDIC reference in double precision and calls back into SystemVerilog once, to prove that `context` and `export` work.

```

1 #include <math.h>
2 #include <stdio.h>
3 #include "svdpi.h"
4
5 /* The simulator may compile this file with a C++ compiler (Verilator does),

```

Table 2.2. DPI type mapping (IEEE 1800-2017, Annex H). `svBitVecVal` and `svLogicVecVal` are arrays of 32-bit chunks.

SystemVerilog	C	Notes
byte	char	8-bit, two-state
shortint	short int	16-bit
int	int	32-bit
longint	long long	64-bit
real	double	
shortreal	float	
string	const char*	read-only, borrowed
chandle	void*	opaque pointer
bit	svBit	one two-state bit
logic, reg	svLogic	one four-state bit
bit [N-1:0]	svBitVecVal*	packed, 2-state
logic [N-1:0]	svLogicVecVal*	packed, 4-state: aval+bval
open array []	svOpenArrayHandle	use the <code>svSize/svGetArrElemPtr</code> API

```

6   so every DPI symbol must have C linkage in both cases. */
7   #ifdef __cplusplus
8   extern "C" {
9   #endif
10
11  extern void sv_note(const char* msg); /* exported from SystemVerilog */
12
13  #define QF 14
14  #define QA 13
15  #define NIT 14
16
17  static double cordic_gain(void) { /* K = prod sqrt(1 + 2-2i) */
18      double k = 1.0;
19      for (int i = 0; i < NIT; i++) k *= sqrt(1.0 + pow(2.0, -2.0 * i));
20      return k;
21  }
22  double c_cordic_k(void) { return cordic_gain(); }
23
24  void c_cordic_ref(int x0, int y0, int theta, double* xr, double* yr) {
25      static int first = 1;
26      double th = (double)theta / (double)(1 << QA);
27      double k = cordic_gain();
28      *xr = k * ((double)x0 * cos(th) - (double)y0 * sin(th));
29      *yr = k * ((double)x0 * sin(th) + (double)y0 * cos(th));
30      if (first) {
31          char buf[128];
32          snprintf(buf, sizeof buf, "C reference active, K = %.9f", k);
33          sv_note(buf); /* needs `context' on the import */
34          first = 0;
35      }
36  }
37
38  /* 4-state vs 2-state: `logic' arrives as svLogicVecVal (aval + bval),
39   `bit' as svBitVecVal (one word). Getting this wrong is silent. */
40  void c_show_types(const svLogicVecVal* l, const svBitVecVal* b) {
41      printf("[C] logic: aval=0x%08x bval=0x%08x bit: 0x%08x\n",
42             (unsigned)l->aval, (unsigned)l->bval, (unsigned)*b);
43  }
44
45  #ifdef __cplusplus
46  }

```

```
47 | #endif
```

Listing 2.9. The C reference model (`cordic_dpi.c`).

Verilator compiles the C file as part of the build:

```
$ verilator --binary --timing --top-module tb_dpi \
    sv/cordic_pkg.sv sv/tb_dpi.sv cpp/cordic_dpi.cpp \
    -o Vtb_dpi --Mdir obj_dpi
$ ./obj_dpi/Vtb_dpi

[SV] K from C = 1.646760254
[C] logic: aval=0xdeadbeef bval=0x00000000 bit: 0xdeadbeef
[C] logic: aval=0x0000beef bval=0x00000000 bit: 0x0000beef
[SV] note from C: C reference active, K = 1.646760254
[SV] worst |model - C reference| = 3.597 LSB over 361 angles
```

△ Trap: `logic` and `bit` are different C types

The two `c_show_types` calls in the run above passed `32'hDEAD_BEEF` and then `32'hXXXX_BEEF`. A `logic` argument arrives as an `svLogicVecVal`, a struct with an `aval` and a `bval` word: `bval` bits mark the positions that are X or Z. A `bit` argument arrives as a plain `svBitVecVal`, one word, with no way to represent X. If you declare the C parameter as `svBitVecVal*` but the SystemVerilog side passes a `logic`, you will read the `aval` word and silently treat every X as whatever the simulator happened to leave there.

Note also what Verilator printed: `bval` is zero in both calls, and the second call shows `aval=0x0000beef`. Verilator is a two-state simulator, so the X nibbles became zeros before the call was made. On a four-state simulator the second line would show a non-zero `bval`. Do not use Verilator to test four-state DPI behaviour.

2.5.3 The same thing on QuestaSim

QuestaSim is not installed on the machine this chapter was written on, so the following command lines are given as reference and are *not* verified here. They are the standard three steps: compile the SystemVerilog, build a shared object, load it at elaboration.

```
# Not executed for this book -- QuestaSim was unavailable.
vlib work
vlog -sv sv/cordic_pkg.sv sv/tb_dpi.sv
gcc -shared -fPIC -I$Questa_HOME/include \
    cpp/cordic_dpi.c -o cordic_dpi.so -lm
vsim -c -sv_lib cordic_dpi tb_dpi -do "run -all; quit"
```

Note that `-sv_lib` takes the library name *without* the `.so` suffix; passing `cordic_dpi.so` is the most common first mistake. `vsim` also accepts `-dpicpppath` to select the compiler it uses when it builds the shim itself.

2.5.4 Embedding CPython in the shim

Now the step that motivates this section: instead of reimplementing the model in C, have the C shim call the Python one. CPython can be embedded in any process. Listing 2.10 is a working shim.

```
1 | #define PY_SSIZE_T_CLEAN
2 | #include <Python.h>
3 | #include "svdpi.h"
```

```

4
5 extern "C" {
6
7 static PyObject* g_fn = nullptr; /* cordic_model.cordic_rot */
8
9 void py_ref_open(const char* module, const char* func) {
10     if (!Py_IsInitialized()) {
11         PyConfig cfg;
12         PyConfig_InitPythonConfig(&cfg);
13         Py_InitializeFromConfig(&cfg);
14         PyConfig_Clear(&cfg);
15         PyRun_SimpleString("import sys; sys.path.insert(0, 'py')");
16     }
17     PyObject* m = PyImport_ImportModule(module);
18     if (!m) { PyErr_Print(); return; }
19     g_fn = PyObject_GetAttrString(m, func);
20     Py_DECREF(m);
21     if (!g_fn || !PyCallable_Check(g_fn)) { PyErr_Print(); g_fn = nullptr; }
22 }
23
24 void py_ref_call(int x0, int y0, int th, int* xo, int* yo) {
25     *xo = 0; *yo = 0;
26     if (!g_fn) return;
27     PyGILState_STATE gil = PyGILState_Ensure(); /* one interpreter, one GIL */
28     PyObject* r = PyObject_CallFunction(g_fn, "iii", x0, y0, th);
29     if (!r) { PyErr_Print(); PyGILState_Release(gil); return; }
30     *xo = (int)PyLong_AsLong(PyTuple_GetItem(r, 0));
31     *yo = (int)PyLong_AsLong(PyTuple_GetItem(r, 1));
32     Py_DECREF(r);
33     PyGILState_Release(gil);
34 }
35
36 void py_ref_close(void) { Py_XDECREF(g_fn); g_fn = nullptr; Py_FinalizeEx(); }
37
38 } /* extern "C" */

```

Listing 2.10. A DPI shim with CPython embedded (`py_dpi.cpp`).

Built against the system CPython and run under Verilator:

```

$ verilator --binary --timing --top-module tb_pydpi \
  -CFLAGS "$(python3.11-config --includes)" \
  -LDFLAGS "$(python3.11-config --embed --ldflags)" \
  sv/cordic_pkg.sv sv/tb_pydpi.sv cpp/py_dpi.cpp \
  -o Vtb_pydpi --Mdir obj_pydpi
$ ./obj_pydpi/Vtb_pydpi

```

```

[SV] theta=0 -> python says (16383, 1)
[SV] theta=2458 -> python says (15651, 4841)
[SV] theta=4915 -> python says (13521, 9251)
[SV] theta=7373 -> python says (10183, 12834)
[SV] theta=9830 -> python says (5936, 15270)

```

It works. Now the costs, measured on the same machine, 50 000 calls of each:

```

[bench] 50000 calls: embedded Python 0.818 s, C 0.018 s, ratio 46.7

```

About 16 μ s per Python call against 0.36 μ s for the C one – between one and two orders of magnitude, and that is for a model whose body is a fourteen-iteration integer loop. The gap is mostly call overhead: argument marshalling, tuple allocation, the GIL. A model doing real numpy work per call would look better in relative terms and worse in absolute ones.

The other costs are not measurable but matter more:

- **Interpreter start-up** happens once, but it happens inside the simulator, and if it fails the simulator dies with a stack trace nobody can read.
- **The GIL.** One interpreter per process. If your simulator is multi-threaded (Verilator with `-threads`, or a commercial simulator running a multi-core partition) every DPI call into Python serialises.
- **One process, one crash.** A segfault in a C extension module takes the simulation with it, mid-run, with no waveform flushed.
- **Fragile builds.** The simulator, the Python interpreter and every extension module must agree on the C++ ABI and the C runtime. Mixing a conda Python with a vendor simulator's bundled libstdc++ is a well-known way to lose an afternoon.

2.5.5 The pragmatic alternative

If you already have the Python model and you want it in the loop, a long-lived Python worker process talking to the simulator over a pipe or a socket avoids every one of those problems except speed. The shim in C stays tiny, the interpreter lives in its own process with its own crash domain, and you can restart or debug the model independently. That is coupling style (d), and Section 2.8 shows it.

Do not embed CPython in a simulator for a student project or a thesis. Use offline vectors, generated source, or cocotb. Embed CPython only when you have an existing Python model that must be in the loop, no cocotb-supported simulator, and a build engineer.

2.6 In-process, part 2: Verilator and pybind11

Verilator compiles your design into a C++ class. A C++ class can be wrapped for Python with pybind11 in about forty lines. The result is a testbench written entirely in Python, driving a compiled model directly, with no VPI, no DPI and no simulator kernel in between.

```

1  #include <pybind11/pybind11.h>
2  #include "Vcordic_rot.h"
3  #include "verilated.h"
4
5  namespace py = pybind11;
6
7  class Cordic {
8  public:
9      Cordic() : ctx_(new VerilatedContext), dut_(new Vcordic_rot(ctx_)) {}
10     ~Cordic() { dut_>final(); delete dut_; delete ctx_; }
11
12     // One half cycle: set the clock, evaluate, advance simulation time.
13     void half_tick(int level) {
14         dut_>clk_i = level;
15         dut_>eval();
16         ctx_>timeInc(5000); // 5 ns; timeInc counts timeprecision units
17     }
18     void tick() { half_tick(0); half_tick(1); }
19
20     void set_rst(int v) { dut_>rst_ni = v; }
21     void set_valid(int v) { dut_>req_valid_i = v; }
22     void set_ready(int v) { dut_>rsp_ready_i = v; }

```

```

23 void set_req(int x, int y, int th) {
24     dut->req_x_i    = static_cast<uint16_t>(x);
25     dut->req_y_i    = static_cast<uint16_t>(y);
26     dut->req_theta_i = static_cast<uint16_t>(th);
27 }
28 void eval() { dut->eval(); }
29 bool req_ready() const { return dut->req_ready_o; }
30 bool rsp_valid() const { return dut->rsp_valid_o; }
31 int  rsp_x()  const { return static_cast<int16_t>(dut->rsp_x_o); }
32 int  rsp_y()  const { return static_cast<int16_t>(dut->rsp_y_o); }
33 uint64_t time() const { return ctx->time(); }
34
35 private:
36     VerilatedContext* ctx_;
37     Vcortic_rot* dut_;
38 };
39
40 PYBIND11_MODULE(vcortic, m) {
41     m.doc() = "Verilated cordic_rot, driven from Python";
42     py::class_<Cortic>(m, "Cortic")
43         .def(py::init<>())
44         .def("tick", &Cortic::tick).def("eval", &Cortic::eval)
45         .def("set_rst", &Cortic::set_rst)
46         .def("set_valid", &Cortic::set_valid)
47         .def("set_ready", &Cortic::set_ready)
48         .def("set_req", &Cortic::set_req)
49         .def_property_readonly("req_ready", &Cortic::req_ready)
50         .def_property_readonly("rsp_valid", &Cortic::rsp_valid)
51         .def_property_readonly("rsp_x", &Cortic::rsp_x)
52         .def_property_readonly("rsp_y", &Cortic::rsp_y)
53         .def_property_readonly("time", &Cortic::time);
54 }

```

Listing 2.11. A pybind11 wrapper around the Verilated model (pybind_cortic.cpp).

Recipe: Building the extension

Verilator must be told to compile position-independent code, because the result goes into a shared object. Forgetting `-CFLAGS -fPIC` gives a linker error about `R_X86_64_TPOFF32` relocations that looks far more alarming than it is.

```

$ verilator --cc --top-module cordic_rot -CFLAGS -fPIC \
    sv/cortic_pkg.sv sv/cortic_rot.sv --Mdir obj_pb
$ make -C obj_pb -f Vcortic_rot.mk
$ g++ -O2 -std=c++17 -shared -fPIC \
    $(python3 -m pybind11 --includes) \
    -I obj_pb -I /usr/share/verilator/include \
    -I /usr/share/verilator/include/vltstd \
    cpp/pybind_cortic.cpp obj_pb/libVcortic_rot.a obj_pb/libverilated.a \
    -o vcortic.so

```

The driver is then ordinary Python, with numpy available and the golden model one import away.

```

1 import numpy as np
2 import cordic_model as m
3 import vcortic
4
5 def run(dut, x0, y0, th):
6     dut.set_req(x0, y0, th)
7     dut.set_valid(1)

```

```

8     dut.eval()
9     while not dut.req_ready:
10         dut.tick()
11     dut.tick()                                # the accepting edge
12     dut.set_valid(0)
13     dut.eval()
14     while not dut.rsp_valid:
15         dut.tick()
16     return dut.rsp_x, dut.rsp_y
17
18 dut = vcordic.Cordic()
19 dut.set_rst(0); dut.set_ready(1)
20 for _ in range(3):
21     dut.tick()
22 dut.set_rst(1)
23
24 angles = np.linspace(-np.pi, np.pi, 721)
25 thetas = [m.float_to_q(a, m.QA, m.AW) for a in angles]
26 got = [run(dut, m.XO_UNIT, 0, th) for th in thetas]
27 exp = [m.cordic_rot(m.XO_UNIT, 0, th) for th in thetas]
28 print("bit-exact misses :",
29       sum(1 for g, e in zip(got, exp) if g != e))
30
31 gx = np.array([g[0] for g in got]) / float(1 << m.QF)
32 gy = np.array([g[1] for g in got]) / float(1 << m.QF)
33 err = np.maximum(np.abs(gx - np.cos(angles)), np.abs(gy - np.sin(angles)))
34 print(f"worst |err| vs sin/cos : {err.max() * (1 << m.QF):.2f} LSB")

```

Listing 2.12. Driving the Verilated model from Python (`drive_pybind.py`).

```

$ python3 py/drive_pybind.py
transactions      : 721
bit-exact misses  : 0
worst |err| vs sin/cos : 3.82 LSB
simulated time    : 115380000 ps (115.38 us)
wall time         : 8.0 ms (11.1 us per transaction)

```

`ctx->time()` counts `timeprecision` units, and the design declares `'timescale 1ns/1ps`, so the raw number is picoseconds. That is why `half_tick()` advances by 5000 and not by 5: see the discussion in Chapter 1.

This is attractive and it is genuinely useful for algorithmic blocks. The whole testbench is Python; `numpy` and `matplotlib` are right there; there is no VPI, no DPI, no simulator licence.

The limits are equally real. You are restricted to Verilator's synthesisable subset, so no `initial` blocks with delays, no classes, no constrained random inside the design, and only partial four-state support. You are writing the clocking yourself: every `eval()` and every `timeInc()` is your responsibility, and a missing `eval()` after changing an input is a bug that looks like a design bug. There is no scheduler, so two "concurrent" drivers are two Python loops you have to interleave by hand. And the wrapper is code you maintain: add a port to the DUT and you edit C++.

For a single-clock, valid/ready block this is fine. For anything with more than two independent interfaces, use `cocotb` instead – which gives you the same Python, plus a scheduler.

2.7 In-process, part 3: cocotb

`cocotb` (coroutine co-simulation testbench) is the option most students should reach for. It is a Python library that drives a DUT through the simulator's own procedural interface – VPI

for Verilog, VHPI or FLI for VHDL – and it replaces the HDL testbench entirely. There is no `tb_top.sv`. The top level of the simulation *is* the DUT, and everything above it is Python.

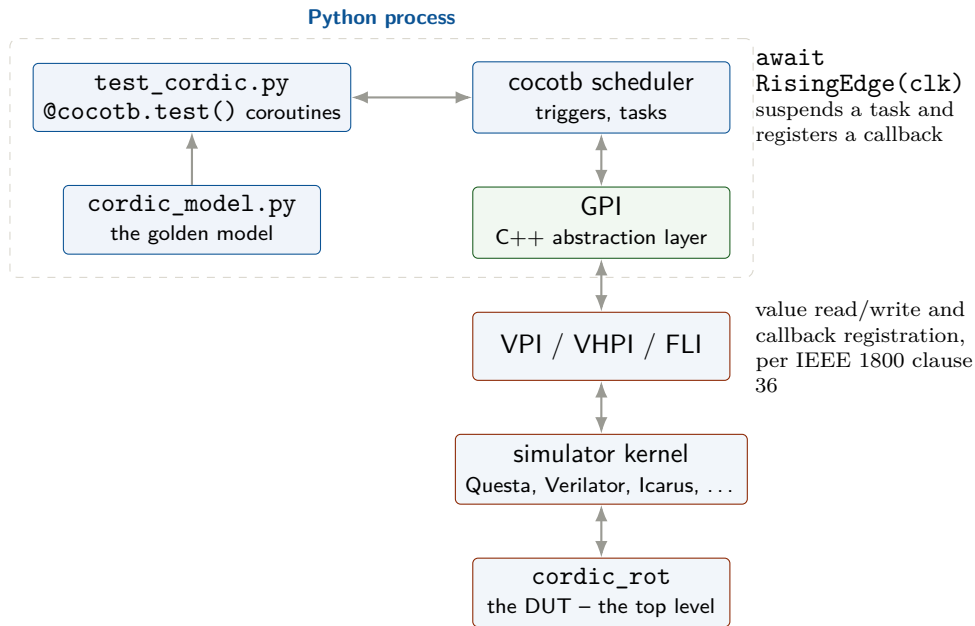


Figure 2.2. cocotb architecture. The Python test and the simulator run in one process; the scheduler suspends coroutines on simulator callbacks registered through the GPI, which is a thin C++ layer over VPI, VHPI or FLI. There is no HDL testbench.

Why there is no HDL testbench

VPI (the Verilog Procedural Interface, IEEE 1800-2017 clause 36) and VHPI (IEEE 1076-2008 clause 22) let an external C program walk the design hierarchy, read and write signal values, and register callbacks on events such as "value changed" or "end of time step". That is exactly what a driver and a monitor need, so a testbench written against VPI does not need to be instantiated inside the design at all. cocotb wraps VPI, VHPI and Mentor's FLI behind one C++ abstraction called the GPI, and exposes it to Python. The consequence for you is practical: the `TOPLEVEL` in a cocotb run is the DUT itself, and there is no HDL file to write.

2.7.1 The core API

Everything in cocotb is a coroutine that awaits a trigger.

- `@cocotb.test()` marks a coroutine as a test. Every marked coroutine in the module named by `MODULE` is run, in order, and each gets the DUT handle as its only argument.
- `await Timer(10, unit="ns")` advances simulation time.
- `await RisingEdge(dut.clk_i)` suspends until the next rising edge. `FallingEdge`, `Edge` (either) and `ValueChange` are the siblings.
- `await ClockCycles(dut.clk_i, 5)` is five rising edges.
- `cocotb.start_soon(coro())` launches a background task and returns immediately – this is how you get a driver and a monitor running concurrently. It is the cocotb equivalent of `fork ... join_none`.
- `Clock(dut.clk_i, 10, unit="ns").start()` is a coroutine that generates a clock; start it

with `start_soon`.

- `await Combine(a, b)` waits for all of several tasks; `await First(a, b)` waits for the first of them – `join` and `join_any` respectively.
- `await with_timeout(task, 1000, "ns")` raises `SimTimeoutError` if the task has not finished in time. Use it on every driver: a hung handshake should fail the test, not hang the regression.
- `dut.sig.value = 3` schedules a write; `dut.sig.value` reads. The read gives a `LogicArray` with `.integer` (unsigned), `.signed_integer` (two's complement), `.binstr` (the string of 0, 1, x and z) and `.is_resolvable`.

2.7.2 A complete testbench for the CORDIC

Listing 2.13 is the whole thing: driver, monitor, scoreboard and three tests, importing the golden model directly. Compare it with the class-based SystemVerilog testbench of the *Verification Cookbook*: the structure is identical – transaction, driver, monitor, scoreboard – and the line count is about a fifth.

```

1  import math, os, random, sys
2  from collections import deque
3  from pathlib import Path
4
5  import cocotb
6  from cocotb.clock import Clock
7  from cocotb.triggers import ClockCycles, RisingEdge, with_timeout
8
9  sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "py"))
10 import cordic_model as m
11
12 TOL_LSB = 12
13
14
15 class Txn:
16     """The transaction: stimulus in, observed response filled in later."""
17     def __init__(self, x0, y0, theta):
18         self.x0, self.y0, self.theta = x0, y0, theta
19         self.x = self.y = None
20
21     def __repr__(self):
22         return f"Txn(x0={self.x0}, y0={self.y0}, theta={self.theta})"
23
24
25 async def reset(dut, cycles=5):
26     dut.rst_ni.value = 0
27     dut.req_valid_i.value = 0
28     dut.rsp_ready_i.value = 1
29     dut.req_x_i.value = dut.req_y_i.value = dut.req_theta_i.value = 0
30     await ClockCycles(dut.clk_i, cycles)
31     dut.rst_ni.value = 1
32     await RisingEdge(dut.clk_i)
33
34
35 async def driver(dut, queue, stimulus):
36     """Push transactions into the request channel, honouring req_ready_o."""
37     for txn in stimulus:
38         await RisingEdge(dut.clk_i)
39         dut.req_x_i.value = txn.x0 & 0xFFFF
40         dut.req_y_i.value = txn.y0 & 0xFFFF
41         dut.req_theta_i.value = txn.theta & 0xFFFF
42         dut.req_valid_i.value = 1

```

```

43     while True:
44         await RisingEdge(dut.clk_i)
45         if dut.req_ready_o.value == 1:
46             break
47     dut.req_valid_i.value = 0
48     queue.append(txn)
49
50
51 async def monitor(dut, queue, results, expected):
52     """Sample the response channel on every accepted handshake."""
53     while len(results) < expected:
54         await RisingEdge(dut.clk_i)
55         if dut.rsp_valid_o.value == 1 and dut.rsp_ready_i.value == 1:
56             txn = queue.popleft()
57             txn.x = dut.rsp_x_o.value.signed_integer
58             txn.y = dut.rsp_y_o.value.signed_integer
59             results.append(txn)
60
61
62 def scoreboard(results, log):
63     """Bit-exact check against cordic_model, plus a float sanity check."""
64     errors, worst = 0, 0.0
65     for txn in results:
66         ex, ey = m.cordic_rot(txn.x0, txn.y0, txn.theta)
67         if (txn.x, txn.y) != (ex, ey):
68             errors += 1
69             if errors <= 5:
70                 log.error("%r: got (%d, %d) expected (%d, %d)",
71                           txn, txn.x, txn.y, ex, ey)
72             fx, fy = m.cordic_float(txn.x0, txn.y0, txn.theta / (1 << m.QA))
73             worst = max(worst, abs(txn.x - fx), abs(txn.y - fy))
74     return errors, worst
75
76
77 async def run(dut, stimulus):
78     cocotb.start_soon(Clock(dut.clk_i, 10, unit="ns").start())
79     await reset(dut)
80
81     queue, results = deque(), []
82     mon = cocotb.start_soon(monitor(dut, queue, results, len(stimulus)))
83     drv = cocotb.start_soon(driver(dut, queue, stimulus))
84     await with_timeout(drv, 200 * len(stimulus) + 1000, "ns")
85     await with_timeout(mon, 200 * len(stimulus) + 1000, "ns")
86
87     errors, worst = scoreboard(results, dut._log)
88     dut._log.info("%d transactions, %d bit-exact mismatches, "
89                  "worst float error %.2f LSB", len(results), errors, worst)
90     assert errors == 0, f"{errors} bit-exact mismatches"
91     assert worst <= TOL_LSB, f"worst float error {worst:.2f} > {TOL_LSB} LSB"
92
93
94 @cocotb.test()
95 async def test_directed(dut):
96     """Angles a random generator will not hit."""
97     angs = [0.0, math.pi/2, -math.pi/2, math.pi/4, -math.pi/4,
98            1e-4, -1e-4, 3.14, -3.14]
99     stim = [Txn(m.X0_UNIT, 0, m.float_to_q(a, m.QA, m.AW)) for a in angs]
100    stim += [Txn(0, 0, 0), Txn(6000, -6000, 0)]
101    await run(dut, stim)
102
103
104 @cocotb.test()
105 async def test_random(dut):

```

```

106     """Constrained random stimulus, seeded from the environment."""
107     seed = int(os.environ.get("SEED", "20250905"))
108     rng = random.Random(seed)
109     lim = m.float_to_q(math.pi, m.QA, m.AW)
110     stim = [Txn(rng.randint(-6000, 6000), rng.randint(-6000, 6000),
111                rng.randint(-lim, lim)) for _ in range(100)]
112     dut._log.info("seed = %d", seed)
113     await run(dut, stim)
114
115
116 # test_sweep is the same shape: 181 angles spread over [-pi, pi].

```

Listing 2.13. A complete cocotb testbench for `cordic_rot` (`test_cordic.py`).

The build is a `Makefile` of a dozen lines. Everything after `include` comes from cocotb.

```

SIM          ?= icarus
TOPLEVEL_LANG ?= verilog

RTL := $(PWD)/../sv

ifeq ($(TOPLEVEL_LANG),verilog)
    VERILOG_SOURCES := $(RTL)/cordic_pkg.sv $(RTL)/cordic_rot.sv
    ifeq ($(SIM),verilator)
        EXTRA_ARGS += --timing -Wno-fatal
    endif
else
    VHDL_SOURCES := $(RTL)/cordic_pkg.vhd $(RTL)/cordic_rot.vhd
endif

TOPLEVEL := cordic_rot
MODULE    := test_cordic

include $(shell cocotb-config --makefiles)/Makefile.sim

```

Four variables carry all the configuration. `SIM` names the simulator; `TOPLEVEL_LANG` is `verilog` or `vhdl` and decides which of `VERILOG_SOURCES` and `VHDL_SOURCES` is used; `TOPLEVEL` is the name of the DUT as the simulator sees it, which is where a case-sensitivity mismatch between VHDL and the makefile usually bites; and `MODULE` is the Python module (not file) containing the tests. Two more are worth knowing: `COMPILE_ARGS` goes to the analysis step and `EXTRA_ARGS` to both analysis and elaboration, while `SIM_ARGS` goes only to the run. The language-revision switch is an *analysis* switch: `-2008` for `vcom` belongs in `COMPILE_ARGS`, and a switch that a tool needs at both analysis and elaboration belongs in `EXTRA_ARGS`. Put one of them in `SIM_ARGS` by mistake and the run either rejects it as an unknown option or, worse, looks for a library built to a different revision and reports that it cannot find your entity — a message that points at the design and not at the command line.

2.7.3 Running it

With Verilator as the back end:

```
$ make -C cocotb_tb SIM=verilator
```

```

0.00ns INFO cocotb Running on Verilator version 5.020 2024-01-01
0.00ns INFO cocotb Running tests with cocotb v1.9.2
0.00ns INFO cocotb.regression Found test test_cordic.test_directed
0.00ns INFO cocotb.regression Found test test_cordic.test_random
0.00ns INFO cocotb.regression Found test test_cordic.test_sweep
0.00ns INFO cocotb.regression running test_directed (1/3)

```

```

1820.00ns INFO cocotb.cordic_rot 11 transactions, 0 bit-exact
                                mismatches, worst float error 1.07 LSB
1820.00ns INFO cocotb.regression test_directed passed
1820.00ns INFO cocotb.regression running test_random (2/3)
1820.00ns INFO cocotb.cordic_rot seed = 20250905
17890.00ns INFO cocotb.cordic_rot 100 transactions, 0 bit-exact
                                mismatches, worst float error 2.06 LSB
17890.00ns INFO cocotb.regression test_random passed
46920.00ns INFO cocotb.cordic_rot 181 transactions, 0 bit-exact
                                mismatches, worst float error 3.60 LSB
46920.00ns INFO cocotb.regression test_sweep passed
** TEST                        STATUS  SIM TIME (ns)  REAL TIME (s)  RATIO **
** test_cordic.test_directed  PASS      1820.00      0.02  79492 **
** test_cordic.test_random    PASS      16070.00     0.18  89996 **
** test_cordic.test_sweep     PASS      29030.00     0.32  90580 **
** TESTS=3 PASS=3 FAIL=0 SKIP=0      46920.00     0.54  87526 **

```

Now the part that matters most to a VHDL student. The *same Python file*, not one line changed, runs against the VHDL implementation of the same core. Only the makefile variables change:

```
$ make -C cocotb_tb SIM=questa TOPLEVEL_LANG=vhdl COMPILE_ARGS=-2008
```

```

1820.00ns INFO cocotb.cordic_rot 11 transactions, 0 bit-exact
                                mismatches, worst float error 1.07 LSB
1820.00ns INFO cocotb.regression test_directed passed
17890.00ns INFO cocotb.cordic_rot 100 transactions, 0 bit-exact
                                mismatches, worst float error 2.06 LSB
17890.00ns INFO cocotb.regression test_random passed
46920.00ns INFO cocotb.cordic_rot 181 transactions, 0 bit-exact
                                mismatches, worst float error 3.60 LSB
46920.00ns INFO cocotb.regression test_sweep passed
** TESTS=3 PASS=3 FAIL=0 SKIP=0                                **

```

Same test names, same transaction counts, same bit-exact result, same worst error. The wall-clock and the simulated-nanoseconds-per-second figures are the one thing that does change, and they change with the simulator rather than with the test, which is why they are left out above.

That is the argument in one line: you can use cocotb on your VHDL coursework today, and the testbench you write will still be valid when the design is rewritten in SystemVerilog. cocotb officially supports Questa/ModelSim, Xcelium, VCS, Riviera-PRO, Verilator and Icarus Verilog, so whichever tool your department has, the Python side is portable.

△ Trap: .value on a multi-bit signal is unsigned

Reading a port gives you a bit pattern, and cocotb interprets it as an unsigned integer by default – regardless of whether the SystemVerilog declared it **logic signed** or the VHDL declared it **signed**. The extra test below reads `rsp_x_o` after a rotation by π , where the answer should be -16384 :

```

binstr      = 1100000000000000
int(v)      = 49152
v.integer   = 49152
v.signed_integer = -16384
model says  = -16384

```

Use `.signed_integer` for any signed port; if you use `.integer` or `int()`, every negative result will read as a large positive one and your scoreboard will report a hundred per cent

failure rate on a working design. When writing, the reverse applies: `dut.req_x_i.value = -1234` is accepted by recent cocotb, but masking explicitly (`value & 0xFFFF`) is unambiguous and works everywhere.

△ Trap: cocotb and simulator versions are coupled

cocotb talks to the simulator through VPI/VHPI, and both sides evolve. The first attempt at the run above used cocotb 2.1.0 and failed immediately:

```
Makefile.verilator:29: *** cocotb requires Verilator 5.036 or later,
                        but using 5.020. Stop.
```

The run shown was produced with cocotb 1.9.2, which supports Verilator 5.020. Note also that cocotb 2.0 changed the API in ways that matter for this chapter: `units=` became `unit=`, and the value accessors were reorganised. Pin cocotb in your `requirements.txt` together with the simulator version you tested against, and write the pair into the CI job.

2.7.4 The ecosystem

cocotb is deliberately small. Four packages sit on top of it and you should know they exist.

`cocotb-test` (and, in cocotb 2.x, the built-in `pytest` support) replaces the `Makefile` with a `pytest` entry point. That gives you `pytest -k sweep`, parameterised tests, fixtures, parallel runs with `pytest-xdist` and a JUnit XML report that every CI system understands. For a regression of more than a handful of tests it is worth the change.

`cocotb-bus` provides the `Driver`, `Monitor` and `Scoreboard` base classes that used to live in cocotb itself, plus ready-made verification IP for AXI4, AXI4-Lite, AXI-Stream, Avalon and Wishbone. If your DUT has an AXI port, do not write the driver.

`cocotb-coverage` adds constrained random generation and functional coverage in Python: `@CoverPoint` and `@CoverCross` decorators that collect bins as your tests run, and a constraint solver for randomisation. It covers the same ground as SystemVerilog's `covergroup` and `constraint` blocks, in a form that will look familiar after this chapter.

`pyuvvm` is UVM 1.2 reimplemented in Python: `uvm_component`, `uvm_sequence`, `uvm_config_db`, the factory, the phases, TLM ports, all of it, running on cocotb. If you have read the *Verification Cookbook*, `pyuvvm` will be immediately legible, and it is an excellent way to learn UVM without also fighting SystemVerilog class syntax and a licence server. It is not, today, what industry signs off with.

2.7.5 Honest limits

Speed. Every `await RisingEdge(...)` is a callback from the simulator into Python, and a testbench with a driver and a monitor both waiting on the clock costs two Python round-trips per cycle. On this machine the cocotb run above sustained about 97 000 ns of simulated time per wall-clock second. The pure C++ Verilator testbench of Section 2.3, driving the same design over the same handshake, sustained about 42 ms of simulated time per second – some two to three orders of magnitude more. That ratio is workload dependent: a testbench that awaits a transaction rather than a clock edge, on a design with a hundred times more logic, will show a much smaller gap, because the Python overhead is per callback and not per gate. Measure before you conclude anything.

No SVA. You cannot write a concurrent assertion in Python. Assertions that belong to the design should stay in the design, as in `cordic_rot.sv`, and they will still fire during a cocotb run under a simulator that supports them. Verilator supports immediate assertions and a useful subset of concurrent ones; Icarus does not parse `property` at all.

Waveform debug is unchanged. cocotb does not give you a Python waveform viewer. You dump a VCD or FST from the simulator (`EXTRA_ARGS += -trace` for Verilator; a do file with `log -r /*` for QuestaSim) and open it in the simulator’s own wave window, or in GTKWave or Surfer if it is a VCD or an FST — see Section 1.4.4. The Python side is debugged with `logging`, `pdb` or `pytest`.

Industry. Sign-off flows at the companies most of you will join are SystemVerilog and UVM, driven by Questa, VCS or Xcelium. cocotb is widely used for block-level work, for IP developed by small teams, and throughout the open-source silicon community, and it is growing. It is not a reason to skip the *Verification Cookbook*.

2.8 Out-of-process co-simulation

The last style is a small C shim inside the simulator that talks to a Python process over a UNIX socket or a pipe. Use it when the model cannot be linked in: a licensed library that only runs in its own process, a web service, a physical model that takes seconds per call and must not block your build, or simply a Python environment you cannot reconcile with the simulator’s.

The shim is short. Listing 2.14 is the whole client side; the server is thirty lines of Python around `socket.AF_UNIX`.

```

1  static int g_fd = -1;
2
3  int sock_open(const char* path) {
4      struct sockaddr_un a;
5      g_fd = socket(AF_UNIX, SOCK_STREAM, 0);
6      if (g_fd < 0) return -1;
7      std::memset(&a, 0, sizeof a);
8      a.sun_family = AF_UNIX;
9      std::snprintf(a.sun_path, sizeof a.sun_path, "%s", path);
10     if (connect(g_fd, (struct sockaddr*)&a, sizeof a) < 0) return g_fd = -1;
11     return 0;
12 }
13
14 int sock_ref(int x0, int y0, int th, int* xo, int* yo) {
15     char buf[128];
16     int n = std::snprintf(buf, sizeof buf, "%d %d %d\n", x0, y0, th);
17     if (g_fd < 0 || write(g_fd, buf, n) != n) return -1;
18     n = read(g_fd, buf, sizeof buf - 1);      /* blocking: deadlock risk */
19     if (n <= 0) return -1;
20     buf[n] = 0;
21     return (std::sscanf(buf, "%d %d", xo, yo) == 2) ? 0 : -1;
22 }
23
24 void sock_close(void) { if (g_fd >= 0) close(g_fd); g_fd = -1; }
```

Listing 2.14. A DPI shim over a UNIX socket (`sock_dpi.cpp`).

```

$ python3 py/ref_server.py /tmp/cordic.sock &
[srv] listening on /tmp/cordic.sock
$ ./obj_sock/Vtb_sock
[SV] k=0 -> server says (16383, 1) rc=0
[SV] k=1 -> server says (15089, 6381) rc=0
[SV] k=2 -> server says (11413, 11754) rc=0
```

```
[SV] k=3 -> server says (5936, 15270) rc=0
[svr] served 4 requests
```

The failure modes are the classic ones and they will all happen to you.

Deadlock. The `read` above is blocking. If the server crashes, the simulation hangs forever, and in CI that means a job that runs until the runner's timeout. Always set `SO_RCVTIMEO` or use `poll` with a deadline, and fail the test on timeout.

Ordering. A stream socket has no message framing. Two requests issued before the first reply is read will be concatenated in the buffer and your `sscanf` will read the wrong pair. Either keep the protocol strictly request-response (as above), or put an explicit length prefix on every message.

Start-up race. The simulator must not connect before the server has bound the socket. In a `Makefile` that means a wait loop on the socket file, not `sleep 1`.

Cleanup. A stale socket file from a crashed run makes the next run fail with a confusing error. Unlink it on start-up, and put the socket path under a per-run temporary directory.

Socket co-simulation is a last resort. If you can write the model in Python and use cocotb, do that instead; if you can precompute the answers, use offline vectors.

2.9 Choosing

Table 2.3 maps the cases you will actually meet onto a style.

Start with offline vectors. Move to cocotb when the DUT needs stimulus that depends on its own output. Move to DPI-C only when a simulator or a methodology forces you to.

The four styles are not exclusive, and a mature flow usually runs three of them at once. The CORDIC project in this book generates its constants at build time, drives a fast offline-vector regression of a million transactions in a C++ Verilator testbench overnight, and runs a small cocotb suite on every push for the interactive protocol cases – backpressure, reset in the middle of a rotation, an angle out of range. Each style is doing what it is good at, and none of them duplicates the golden model, because all three read `cordic_model.py`.

That last point is the one to take away from the whole chapter. The question is not "which coupling style is best" but "where does the single source of truth live". Put it in one Python file, make everything else import it or be generated from it, and the four styles become four ways of shipping the same model to the same design.

2.10 Putting it in CI

A verification flow that only runs on your laptop is not a flow. The `Makefile` below is the real one used for this chapter: it lints, regenerates vectors when the model or the generator changes, runs the simulation, and post-processes the results in Python. Every target has been executed.

```
PYTHON ?= python3
SEED    ?= 20250905
COUNT ?= 200
RTL     := sv/cordic_pkg.sv sv/cordic_rot.sv
VEC     := vec/cordic_vec.txt vec/cordic_vec.hex vec/cordic_vec.n
```

Table 2.3. Which coupling style to use.

Situation	Use	Why
Homework testbench, one simulator, a few hundred vectors	(a) offline vectors	Nothing to install, the file is the evidence, works everywhere.
Lab exercise in VHDL	cocotb, with <code>TOPLEVEL_LANG</code> set to <code>vhdl</code>	The HDL testbench disappears; the golden model is imported directly.
Constant table or ROM the tools will not compute	(b) generated source	Integer literals are accepted by every tool. Add a <code>gen-check</code> job.
Thesis accelerator, Verilator in CI, algorithmic block	cocotb, or <code>pybind11</code> if you need raw speed	Full <code>numpy</code> in the scoreboard; no licence.
Long random regression, millions of transactions	(a) offline vectors, or a C++ testbench	Python per-cycle callbacks would dominate the run time.
Industrial block, Questa, UVM sign-off	(a) for vectors, DPI-C for a C reference model	UVM is the deliverable; Python stays outside the simulation.
Existing Python or licensed model that must be in the loop	(d) sockets	Crash isolation and environment isolation are worth the pain.

```

GEN      := gen/cordic_tbl.h gen/cordic_tbl_pkg.sv

.PHONY: all lint gen gen-check sim check cocotb

all: lint sim check

lint:
    verilator --lint-only -Wall --top-module cordic_rot $(RTL)

$(VEC): py/gen_vectors.py py/cordic_model.py
    $(PYTHON) py/gen_vectors.py --seed $(SEED) --count $(COUNT) --outdir vec

$(GEN): py/gen_tables.py py/cordic_model.py
    $(PYTHON) py/gen_tables.py gen

gen-check: gen
    $(PYTHON) py/gen_tables.py .gencheck
    diff -r gen .gencheck || \
        (echo "generated files are stale: run 'make gen'"; exit 1)

obj_file/Vtb_file: $(RTL) sv/tb_vec_file.sv
    verilator --binary --timing --top-module tb_vec_file \
        $(RTL) sv/tb_vec_file.sv -o Vtb_file --Mdir obj_file

sim: obj_file/Vtb_file $(VEC)
    ./obj_file/Vtb_file +DIR=vec

check: sim
    $(PYTHON) py/check_results.py vec/cordic_res.txt --tol-lsb 12

cocotb:
    $(MAKE) -C cocotb_tb SIM=verilator
    $(MAKE) -C cocotb_tb SIM=questa TOPLEVEL_LANG=vhdl COMPILE_ARGS=-2008

```

```

$ make gen-check all
python3 py/gen_tables.py .gencheck
diff -r gen .gencheck || (echo "generated files are stale..."; exit 1)
verilator --lint-only -Wall --top-module cordic_rot sv/cordic_pkg.sv \
    sv/cordic_rot.sv
python3 py/gen_vectors.py --seed 20250905 --count 200 --outdir vec
wrote 212 vectors to vec/cordic_vec.{txt,hex}
./obj_file/Vtb_file +DIR=vec
[tb] 217 lines, 212 vectors, 0 mismatches
[tb] PASS
python3 py/check_results.py vec/cordic_res.txt --tol-lsb 12
vectors          : 212
saturated         : 2 (excluded)
worst |error|     : 2.655 LSB
rms  error       : 0.791 LSB
tolerance         : 12.0 LSB
outside tol       : 0

```

The corresponding GitHub Actions job is short because the `Makefile` does the work. Note the pinned versions: this is the whole point of the exercise.

```

name: verify
on: [push, pull_request]

jobs:
  cordic:
    runs-on: ubuntu-24.04
    steps:

```

```

- uses: actions/checkout@v4

- name: Install tools
  run: |
    sudo apt-get update && sudo apt-get install -y verilator
    python3 -m pip install "cocotb==1.9.2" numpy matplotlib jinja2

- name: Generated files are up to date
  run: make gen-check

- name: Lint, vectors, simulation, numerical check
  run: make lint all

- name: cocotb regression
  run: make cocotb

- name: Archive vectors and results
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: cordic-vectors
    path: |
      vec/
      cocotb_tb/results.xml

```

Recipe: Make the build fail, not the log

Every step above fails the job on a mismatch, because `check_results.py` returns a non-zero exit code when a vector is outside tolerance and `make` stops on the first failing recipe. A testbench that prints **FAIL** and exits zero is worse than no testbench: it trains everybody to read logs, and nobody reads logs. Finish every checking script with an explicit `sys.exit(1)`, and every HDL testbench with `$fatal` on error.

Archive the vectors, not just the log

The `upload-artifact` step matters more than it looks. When a nightly regression fails on seed 8842 and nobody can reproduce it, the vector file that produced the failure – header and all – is the difference between a five-minute fix and a week of guessing. Keep it for every failing run.

2.11 Checklist

- Write the reference model in the RTL language if it fits on one screen and uses only arithmetic. Write it in Python as soon as it needs arrays, a transform, a file or a plot.
- Default to **offline vectors**. They work with every simulator, they leave evidence on disk, and they cost nothing at run time.
- A vector file is hex, one transaction per line, with a `//` header carrying the seed, every parameter and the model revision. Print that header from the testbench at time zero.
- Never call `$readmemh` without an explicit address range. It does not tell you how many entries it filled, and a short file passes silently – especially under Verilator, where the unfilled entries are zero rather than `'x'`.
- Check the return value of `$sscanf`, and remember that `$feof` is not a look-ahead: check `$fgets` as well. VHDL's `endfile` is a look-ahead, which is why you forget.

- Hex vector files carry no sign. Every reader must sign-extend explicitly: `$signed` in SystemVerilog, `signed(...)` in VHDL, `static_cast<int16_t>` in C++, an explicit mask in Python.
- Compare bit-exact models with `==`. Compare a floating point reference with a derived tolerance, exclude the cases the reference cannot model (saturation), and never raise the tolerance to make a test pass.
- Every generated artefact needs a Make rule whose prerequisites include the generator and everything it imports, plus a CI job that regenerates and diffs.
- Emit a **package**, not an `'include`. One elaboration, one copy, one name.
- Generate a constant table in Python when any tool in your flow rejects the elaboration-time version. Verilator 5.020 accepts `$atan` in a constant function; Icarus 12.0 does not.
- In DPI-C: **pure** means no side effects and no callback; **context** is required to call an exported function or to use `svGetScope`. **logic** maps to `svLogicVecVal (aval+bval)` and **bit** to `svBitVecVal`. Give every DPI symbol C linkage.
- Do not embed CPython in a simulator unless you have no alternative. It costs one to two orders of magnitude per call, one GIL, one crash domain and a fragile build.
- Use **cocotb** when the stimulus must react to the DUT, and for VHDL coursework in particular: the HDL testbench disappears and the Python one survives a rewrite of the design into SystemVerilog.
- In cocotb, read signed ports with `.signed_integer`, never `.integer` or `int()`. Wrap every driver in `with_timeout`.
- Pin cocotb and the simulator version together. cocotb 1.9.2 works with Verilator 5.020; cocotb 2.1.0 requires 5.036 or later and renamed `units=` to `unit=`.
- Sockets are the last resort. If you use them, frame the messages, set a receive timeout, and unlink the socket file on start-up.

Command-Line Reference

The command lines used throughout this booklet, gathered onto one page: QuestaSim, Verilator, Icarus Verilog, and cocotb.

A.1 Command lines

QuestaSim

```
vlib work
vlog -sv +incdir+. cordic_pkg.sv cordic_if.sv cordic_rot.sv \
    tb_cordic_pkg.sv tb_cordic_top.sv
vsim -c -sv_seed random -do "run -all; quit" tb_cordic_top

# with UVM: compile the library too, and run the UVM top
vlog -sv -uvm +incdir+. cordic_pkg.sv cordic_if.sv cordic_rot.sv \
    cordic_uvm_pkg.sv tb_cordic_uvm.sv
vsim -c -uvm +UVM_TESTNAME=cordic_random_test -sv_seed 1234 \
    -do "run -all; quit" tb_cordic_uvm
```

Verilator

```
# lint only, the first thing to run on any new file
verilator --lint-only -Wall cordic_pkg.sv cordic_rot.sv --top-module cordic_rot

# C++ testbench
verilator --cc --exe --build --trace -j 0 \
    cordic_pkg.sv cordic_rot.sv tb_cordic.cpp --top-module cordic_rot
./obj_dir/Vcordic_rot

# SystemVerilog testbench, run directly
verilator --binary --timing -j 0 cordic_pkg.sv cordic_rot.sv tb.sv
./obj_dir/Vtb
```

QuestaSim, VHDL

```
vlib work
vcom -2008 cordic_pkg.vhd cordic_rot.vhd tb_cordic_rot.vhd
vsim -c -gN_RANDOM=300 -do "run -all; quit" tb_cordic_rot

# in a do file, to silence the numeric_std metavalues warnings at time zero
set NumericStdNoWarnings 1
```

Icarus Verilog

```
iverilog -g2012 -o sim cordic_pkg.sv cordic_rot.sv tb.sv && ./sim
```

cocotb

```
# VHDL: the language revision is an ANALYSIS switch, so COMPILE_ARGS.
SIM           ?= questa
TOPLEVEL_LANG ?= vhdl
VHDL_SOURCES  = $(PWD)/cordic_pkg.vhd $(PWD)/cordic_rot.vhd
COMPILE_ARGS  += -2008
TOPLEVEL      = cordic_rot
MODULE        = test_cordic
include $(shell cocotb-config --makefiles)/Makefile.sim

# For SystemVerilog: SIM=questa, or SIM=verilator with Verilator 5.036 or
# later (cocotb refuses anything older).  Icarus cannot compile cordic_pkg.sv
# at all, because of the constant-function ROM.
```

Setting Up a Toolchain

What to install on your own machine so that nothing in this book requires a licence, and what changes when you do have one.

B.1 The free toolchain

Everything in Chapter 1, Chapter 2 and most of the *Example Cookbook* runs on the following, all of which install from a package manager on any recent Linux distribution and work under WSL2 on Windows.

Tool	What it is	Why you want it
Verilator	SystemVerilog to C++ compiler	fast simulation, and the best free linter for SystemVerilog by a wide margin
Icarus Verilog	event-driven Verilog simulator	four-state, event-driven semantics closest to a commercial simulator; partial SystemVerilog
GTKWave or Surfer	waveform viewer	reads VCD, FST and GHW
cocotb	Python testbench framework	drives any of the above from Python
Python 3 with numpy	golden models	
GNU Make, g++	build	Verilator needs a C++17 compiler

B.1.1 Debian, Ubuntu, WSL2

```
sudo apt update
sudo apt install -y verilator iverilog gtkwave \
    build-essential python3 python3-pip python3-numpy
pip3 install --user cocotb cocotb-bus pytest
```

Check the versions. Distribution packages lag: Verilator in particular gained a great deal between 4.x and 5.x, and this book assumes 5.x throughout.

```
verilator --version    # 5.0 or later
iverilog -V | head -1
cocotb-config --version
```

Recipe: Build Verilator from source if the packaged version is old

Verilator builds in a few minutes and is worth building. Clone the repository, check out a release tag, then `autoconf && ./configure && make -j && sudo make install`. The 4.x series does not support `-binary`, `-timing`, classes or virtual interfaces, so half of Chapter 1 will not work on it.

B.1.2 macOS

```
brew install verilator icarus-verilog gtkwave
pip3 install cocotb
```

B.1.3 Windows

Use WSL2 with an Ubuntu image and follow the Debian instructions. Native Windows builds of these tools exist, but the surrounding flow – make, shell scripts, cocotb – is much less painful under WSL.

B.2 The commercial toolchain

B.2.1 QuestaSim / ModelSim

Your department almost certainly has a licence, and Intel distributes a free ModelSim edition with a size limit that is generous enough for anything in this book. The one thing to check is that your version supports the SystemVerilog and UVM features you need: UVM 1.2 support has been standard since roughly 10.x, and IEEE 1800.2 since 2020.x.

```
which vlog vsim vlib
vlog -version
```

B.2.2 Getting UVM

Three options, in decreasing order of convenience.

1. Use the simulator's built-in copy: `vlog -sv +define+UVM_NO_DPI` is not needed if you compile with `-uvm` or set `-uvmhome` to the vendor's bundled tree.
2. Download the Accellera reference implementation, set `UVM_HOME`, and compile `$UVM_HOME/src/uvm_pkg.sv` with `+incdir+$UVM_HOME/src`. Add the DPI library if you want regular expression matching in the config database.
3. `pyuvm`, if you would rather write UVM in Python. See Chapter 2.

B.3 A project layout that works

The layout used by the accompanying sources, and a reasonable default for a thesis or a lab project. It is organised by *language*, because the same design exists here in two of them, and every chapter of this book that names a file names it with one of these prefixes.

```
code/cordic/
sv/                               synthesisable SystemVerilog, the interface,
                                the class testbench
  cordic_pkg.sv cordic_rot.sv cordic_if.sv
  tb_cordic_pkg.sv tb_cordic_top.sv
  cordic_atan_rom.svh cordic_vectors.svh (generated)
vhdl/                             synthesisable VHDL and the VHDL testbench
  cordic_pkg.vhd cordic_rot.vhd tb_cordic_rot.vhd
verilator/                         C++ testbenches and the svtlm.h mini-framework
  tb_cordic.cpp tb_vectors.cpp main_uvmlike.cpp
  cordic_env.h svtlm.h cordic_vectors.h (generated)
python/                           golden model, vector generator, cocotb tests
```

```

    cordic_model.py  gen_vectors.py  test_cordic.py
    Makefile.cocotb
vectors/              generated, safe to delete
    cordic_vectors.txt
build/               build outputs, not committed
Makefile

```

Two things are worth copying. Generated files sit next to the files that consume them and carry a `GENERATED` banner in their first line, so nobody edits one by hand. Everything under `vectors/` and `build/` can be deleted at any moment and rebuilt with `make vectors`.

If you prefer to split by role rather than by language, the usual alternative is `rtl/` for synthesisable sources and `tb/` for everything else. Either works. What does not work is mixing them, so decide on the first day.

B.4 Continuous integration

The single most useful thing you can add to a student project, and it takes ten lines.

```

name: rtl
on: [push, pull_request]
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: sudo apt-get update && sudo apt-get install -y verilator iverilog
      - run: pip install cocotb
      - run: make lint
      - run: make sim

```

The `lint` target should be `verilator -lint-only -Wall` over every synthesisable file, and it should fail the build. Treat a lint warning the way you treat a VHDL analysis error, because in this language that is what it is.

B.5 Editor support

Verible provides a formatter and a language server for SystemVerilog; `rust_hdl` does the same for VHDL. Both integrate with any editor that speaks LSP. A formatter matters more in SystemVerilog than in VHDL because the language gives you so much more freedom to lay code out badly.

Further Reading

Standards

- IEEE Std 1800-2017, *IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language*. The language reference manual. Chapter 9 (processes), Chapter 10 (assignment statements) and Chapter 4 (scheduling semantics) are the three a VHDL designer should read first.
- IEEE Std 1076-2008, *IEEE Standard VHDL Language Reference Manual*, and IEEE Std 1076-2019 for the newer features discussed in the *Design Cookbook*.
- IEEE Std 1800.2-2020, *Universal Verification Methodology Language Reference Manual*.
- IEEE Std 1364-2005, the last standalone Verilog standard, useful only when reading legacy code.

Books

- C. Spear and G. Tumbush, *SystemVerilog for Verification*, 3rd ed., Springer. The standard text on the class-based testbench, and the natural sequel to the *Verification Cookbook*.
- S. Sutherland, S. Davidmann and P. Flake, *SystemVerilog for Design*, 2nd ed., Springer. The design-side counterpart, written by people who were in the room when the language was standardised.
- D. Thomas, *The Verilog Hardware Description Language*, and P. Ashenden, *The Designer's Guide to VHDL*. The two classics, for when you want to know why something is the way it is.
- P. Ashenden and J. Lewis, *VHDL-2008: Just the New Stuff*. Short, and it covers most of what the *Design Cookbook* says VHDL can do.
- M. Keating, *The Simple Art of SoC Design*. Not about either language, and worth reading anyway.

Papers and application notes

- C. Cummings, *Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill!*, SNUG 2000. The clearest published explanation of the blocking/non-blocking rules and the races they exist to prevent. Everything else by the same author on `always_comb`, `unique/priority` and clock-domain crossing is worth the time.
- J. E. Volder, *The CORDIC Trigonometric Computing Technique*, IRE Transactions on Electronic Computers, 1959. The original paper behind the *Example Cookbook*.
- J. S. Walther, *A Unified Algorithm for Elementary Functions*, Spring Joint Computer Conference, 1971. The generalisation.
- R. Andraka, *A Survey of CORDIC Algorithms for FPGA Based Computers*, FPGA 1998. The practical implementation reference, including the gain, the convergence range and the quadrant folding used in the *Example Cookbook*.

Tools and online resources

- *Verilator* manual, <https://verilator.org/guide/>. Read the chapters on the language subset and on the C++ API before writing a C++ testbench.
- *cocotb* documentation, <https://docs.cocotb.org/>.
- *GTKWave* (<https://gtkwave.sourceforge.net>) and *Surfer* (<https://surfer-project.org>), the two waveform viewers that read VCD and FST.
- *OSVVM* (<https://osvvm.org>) and *UVVM* (<https://uvvm.org>), the two VHDL verification frameworks referred to throughout the *Design Cookbook* and the *Verification Cookbook*.
- The *lowRISC* and *OpenTitan* SystemVerilog style guide, <https://github.com/lowRISC/style-guides>. The best free statement of what disciplined SystemVerilog looks like, and the source of the naming conventions used in this book.
- *Verible*, <https://github.com/chipsalliance/verible>, for formatting and linting.

Index

- \$Satan, [38](#)
- \$feof, [43](#)
- \$fgets, [43](#)
- \$readmemh, [42](#)
- \$sscanf, [43](#)
- `include, [51](#)
- `ifdef VERILATOR, [19](#)
- assertion
 - concurrent, [30](#)
- CData, [12](#)
- co-simulation
 - socket, [66](#)
- cocotb, [59](#)
 - cocotb-bus, [65](#)
 - cocotb-coverage, [65](#)
 - performance, [65](#)
 - triggers, [60](#)
- code generation, [49](#)
- continuous integration, [67](#)
- coroutine, [23](#)
- coupling
 - co-simulation, [39](#)
 - generated source, [39](#)
 - in-process, [39](#)
 - offline vectors, [39](#)
- coverage
 - line, [29](#)
 - toggle, [29](#)
- CPython
 - embedding, [55](#)
- delay
 - #, [4](#)
- DPI-C, [52](#)
 - context, [53](#)
 - open array, [53](#)
 - pure, [53](#)
- eval(), [13](#)
- event queue, [2](#)
- factory, [20](#)
- four-state logic, [3](#)
- FST, [15](#)
- gate-level simulation, [5](#)
- golden model, [37](#)
- GTKWave, [15](#)
- high impedance, [4](#)
- hread, [44](#)
- IData, [12](#)
- IP-XACT, [52](#)
- jinja2, [49](#)
- lint, [31](#)
- lint_off, [33](#)
- math_real, [38](#)
- package
 - generated, [51](#)
- PeakRDL, [52](#)
- post-processing, [47](#)
- pybind11, [57](#)
- pyuvvm, [65](#)
- QData, [12](#)
- QuestaSim, [55](#)
- reference model, [37](#)
- reproducibility, [49](#)
- ROM
 - arctangent, [51](#)
- scheduling
 - static, [2](#)
- SData, [12](#)
- SDF, [5](#)
- seed, [49](#)
- stimulus generation, [40](#)
- Surfer, [15](#)
- svGetScope, [53](#)
- textio, [44](#)
- TLM, [20](#)
- tolerance, [48](#)
- VCD, [15](#)
- vector file
 - format, [41](#)
- VerilatedContext, [12](#)
- Verilator, [1](#)

- C++ testbench, [46](#)
- command line, [10](#)
- limitations, [59](#)
- lint, [31](#)
- name, [1](#)
- performance, [8](#)
- verilator_coverage, [30](#)
- VHPI, [60](#)
- VPI, [60](#)
- waveform viewer, [15](#)
- WData, [12](#)
- X propagation, [3](#)